arXiv:2606.21868v2 [cs.LG] 30 Aug 2026

WiSP: A Working-Set View of Mixture-of-Experts Serving on Extremely Low-Resource Hardware

Jiamu Zhang [1] **Liang Wu** [1] **Mayank Darbari** [1] **Liangjie Hong** [1]

Abstract

Modern local and agentic workloads often need large-model capacity at low concurrency, but run on GPUs that cannot keep a frontier-scale model resident. Mixture-of-Experts (MoE) models are a natural fit because they activate only a small subset of experts per token, but their sparsity saves computation, not residency: the full expert pool still has to be stored, and any expert used by a layer must be in GPU memory when that layer runs. Static layer-level CPU offload makes such models fit, but transfers the expert layer in bulk on every forward pass, losing much of the sparsity advantage. We view low-resource MoE serving as a *working-set* problem on the GPU. Routed expert weights and the KV cache are two memory-demand streams competing for the same limited VRAM. We implement this view in **WiSP** (**W**orking-**S**et **P**aging), a routing-aware expert pager that plugs into an unmodified serving engine and preserves byte-identical outputs. On a real 24 GiB RTX 3090, WiSP achieves up to 2.0× the decode throughput of static offload at the same memory budget when the model does not fit. A natural next step is to predict future experts and prefetch them. We find that this does not help in single-stream decode: the bottleneck is PCIe bandwidth, not prediction quality, so speculative transfers compete with demand transfers instead of hiding them. This shifts the design question from prefetching to allocation: how should one VRAM budget be divided between resident experts and the KV cache? We answer with **MV-WSA** (**M**arginal-**V**alue **W**orking-**S**et **A**llocation), which splits memory by marginal latency benefit per byte while enforcing a KV-admission floor. As a startup configurator, MV-WSA is the only policy we test that stays near-best on both prefill and decode; as a live controller, it resizes both pools while serving and reduces end-to-end time by up to 1.19× over a fixed offline split, without changing model outputs. Our code is publicly available at https://github.com/nokia-applied-research/WiSP.

1 Introduction

Many language-model workloads now run outside large datacenters. Local coding assistants, on-device agents, and shared inference on fractional GPUs all need interactive latency at low concurrency, often on hardware that was not provisioned for frontier-scale models. These workloads still benefit from high-capacity models: an agent may switch between code, planning, tool use, and long-context reasoning within one session, so the deployment wants model capacity even when only one or a few requests are active.

Mixture-of-Experts (MoE) models (Shazeer et al., 2017; Fedus et al., 2022) are an attractive fit for this regime because they decouple capacity from per-token computation. Models such as Qwen3-30B-A3B (Qwen Team, 2025), Mixtral-8×7B (Jiang et al., 2024), and DeepSeek-V3 (DeepSeek-AI, 2024) place much of their parameter count in experts, but a token activates only a small routed subset. In principle, this gives a local or agentic deployment access to a large model's capacity without paying dense-model compute on every token.

The obstacle is that sparsity saves computation, but not residency. A token may use only a few experts, but the full expert pool still has to be stored somewhere, and any expert used by a layer must be in GPU memory when that layer runs. Today's MoEs have tens to hundreds of billions of parameters, so commodity GPUs cannot keep the whole model resident even though only a small fraction is active at any step.

A common way to run these models on small GPUs is offloading: keep most weights in host memory and transfer them over PCIe when the forward pass reaches them. This makes the model fit, and its simplest form is static layer-level offload, such as vLLM's `--cpu-offload-gb` (Kwon et al., 2023). Static offload can serve a 30B MoE model in well under 10 GiB of GPU memory, but it treats an MoE layer much like a dense one: when the layer runs, it transfers the expert weights for the layer even though routing will use only a small subset for the actual computation.

[1]Nokia, Sunnyvale, CA 94085, USA. Correspondence to: Jiamu Zhang <jiamu.zhang@nokia.com>.

Preprint. Under Review.

arXiv:2606.21868v2 [cs.LG] 30 Aug 2026

WISP：极低资源硬件上混合专家服务的***working-set***视角

Jiamu Zhang [1] ***Liang Wu*** [1] ***Mayank Darbari*** [1] ***Liangjie Hong*** [1]

摘要

现代本地和代理式工作负载往往需要在低并发下具备大模型容量，但所运行的GPU却无法常驻一个前沿规模的模型。混合专家（MoE）模型是自然契合的方案，因为每个token只激活一小部分专家，但其稀疏性节省的是计算，而不是常驻内存：完整专家池仍必须被存储，而某一层用到的任何专家在该层运行时都必须位于GPU内存中。静态的层级CPU卸载能让此类模型放得下，但会在每次前向传播时批量传输专家层，从而损失掉大部分稀疏性优势。我们将低资源MoE服务视为GPU上的working-set问题。路由专家权重和KV缓存是两股相互竞争同一有限VRAM的内存需求流。我们在WiSP（Working-Set Paging）中实现了这一视角，这是一种路由感知的专家分页器，可接入未经修改的服务引擎，并保持字节级完全一致的输出。在真实的24 GiB RTX 3090上，当模型无法放入显存时，WiSP在相同内存预算下可达到静态卸载最高 2.0× 的解码吞吐量。一个自然的下一步是预测未来专家并进行预取。我们发现这在单流解码中并无帮助：瓶颈是PCIe带宽，而不是预测质量，因此推测性传输会与按需传输竞争，而不是将其隐藏。这将设计问题从预取转向分配：应如何在常驻专家和KV缓存之间划分VRAM预算？我们用MV-WSA（Marginal-Value Working-Set Allocation）来回答，它按每字节边际延迟收益来切分内存，同时强制执行KV准入下限。作为启动配置器，MV-WSA是我们测试过的唯一在预填充和解码上都接近最优的策略；作为运行时控制器，它会在服务过程中动态调整两个池的大小，并在不改变模型输出的情况下，相比固定的离线划分将端到端时间最多减少 1.19× 。我们的代码公开于https://github.com/nokia-applied-research/WiSP。

1 引言

许多语言模型工作负载如今运行在大型数据中心之外。本地编程助手、设备端智能体以及分数GPU上的共享推理，都需要在低并发下实现交互式延迟，且往往运行在并非为前沿规模模型所配置的硬件上。这些工作负载仍然

受益于高容量模型：一个智能体可能在单次会话中在代码、规划、工具使用和长上下文推理之间切换，因此即使只有一个或少数请求处于活跃状态，部署也需要模型容量。

专家混合（MoE）模型（Shazeer 等，2017；Fedus 等，2022）非常适合这一场景，因为它们将容量与每 token 计算量解耦。诸如 Qwen3-30B-A3B（Qwen Team，2025）、Mixtral-8×7B（Jiang 等，2024）和 DeepSeek-V3（DeepSeek-AI，2024）等模型将大部分参数量放在专家中，但一个 token 只激活一小部分被路由的专家。原则上，这使本地或代理式部署能够获得大模型的容量，而无需为每个 token 付出稠密模型的计算代价。

障碍在于：稀疏性节省了计算量，但并未节省驻留内存。一个 token 可能只使用少量专家，但完整的专家池仍必须存储在某个地方，而且某一层用到的任何专家在该层运行时都必须位于 GPU 内存中。如今的 MoE 拥有数百亿到数千亿参数，因此即便在任意步骤中只有一小部分处于活跃状态，消费级 GPU 也无法将整个模型常驻内存。

在小显存 GPU 上运行这些模型的一种常见方式是卸载：将大部分权重保留在主机内存中，并在前向传播到达它们时通过 PCIe 传输。这使得模型能够放入显存，其最简单的形式是静态层级卸载，例如 vLLM 的 --cpu-offload-gb（Kwon 等，2023）。静态卸载可以在远低于 10GiB 的 GPU 显存中服务一个 30B 的 MoE 模型，但它对待 MoE 层的方式与稠密层几乎相同：当该层运行时，它会传输该层的专家权重，尽管路由在实际计算中只会使用其中的一小部分。

[1]诺基亚，美国加利福尼亚州桑尼维尔市 94085。通讯作者：Jiamu Zhang <jiamu.zhang@nokia.com>。

预印本。审阅中。

Several specialized systems improve on this baseline, but they make different tradeoffs. Some keep part of the MoE computation on the CPU, as in ktransformers, which can be fast but relies on Intel AMX (KVCache.AI, 2024), while others target edge deployment by changing the weight format or precision, as in llama.cpp (Gerganov et al., 2023). A third line pipelines, prefetches, or predicts expert transfers to hide communication under computation (Cao et al., 2025; Du et al., 2024; Eliseev & Mazur, 2023). This last line is closest in spirit to WiSP, but its main lever is overlap: it works best when there is enough computation or batching to hide data movement. Our setting is the other end of the spectrum: in low-concurrency interactive serving, each step activates only a small routed subset of experts, but decode exposes little compute behind which to hide PCIe transfers.

The missing piece is therefore not another way to stream the next experts, but a way to decide which experts should remain resident, and how that resident set should share memory with the KV cache.

Serving is a working-set problem. In low-resource MoE serving, the scarce resources are GPU memory and PCIe bandwidth, not arithmetic. This makes the problem look much like the classical *working-set* problem (Denning, 1968; 1970). GPU memory is the small fast store, while host memory is the backing store, and the model generates two streams of memory demand: routed expert weights, determined by the MoE router, and KV-cache blocks, determined by the active tokens. Both have to be resident when used, and both draw from the same VRAM budget.

Under this view, good serving is memory management. The system should keep the experts a workload reuses in GPU memory, leave enough space for the KV blocks of active requests, and avoid admitting more concurrent work than those two working sets can support. This turns expert offload and KV-cache management from two separate knobs into one allocation problem: every byte given to resident experts is a byte taken away from KV, and vice versa. The same abstraction applies to any serving stack with routed experts and an autoregressive KV cache, including MoE LLMs and emerging MoE vision–language models.

The same view also shapes the system design. WiSP changes where weights live and how memory is split, not the computation itself. It leaves the serving engine, kernels, model weights, and numerical precision unchanged, so its outputs are byte-identical to the underlying engine.

WiSP implements this policy as a drop-in plug-in. It reuses the engine's existing expert-map indirection, which maps logical expert ids to physical slots, and makes those slots a small GPU scratch space containing only the resident experts. As the working set changes, WiSP updates the map rather than changing the kernel, model, or precision. This keeps the mechanism architecture-independent for any MoE model the engine can already serve. Section 3 describes the mechanism in detail.

What we find. At matched GPU-memory budgets in the sub-fitting regime, WiSP's routing-aware paging improves decode throughput by up to 2.0× over static offload, measured on a physically constrained 24 GiB RTX 3090 (PCIe 4.0). This is the regime WiSP is designed for: when the model cannot be kept resident, caching only the experts a workload reuses moves far fewer bytes than static layer-level offload. Once enough VRAM is available to keep the model resident, static residency is faster because WiSP's paging indirection becomes unnecessary; we report that crossover explicitly.

We then test the natural next idea: use routing predictions to prefetch experts and hide page faults. In low-concurrency decode, this provides little benefit and can hurt, because there is too little computation behind which to hide a PCIe transfer, and thus decode is bandwidth-bound rather than prediction-bound; speculative transfers mostly compete with demand transfers for the same link. Routing information is therefore more useful as a memory signal—which experts are worth keeping resident—than as a latency signal for prefetching.

This shifts the paper's main question from prefetching to allocation. Expert weights and the KV cache share one VRAM budget: every byte given to resident experts is a byte taken away from KV, and vice versa. We introduce MV-WSA, a marginal-value allocator that splits this budget by equalizing latency benefit per byte, while enforcing a KV-admission floor so the scheduler can still admit the requested concurrency. MV-WSA can be used as a startup configurator, which fixes the split before serving, or as an online controller, which resizes the expert and KV pools while requests are running. In our real-serving allocation test, the configurator is the only policy near-best on both prefill and decode; the online controller further improves end-to-end performance by up to 1.19× over the best offline split at the same VRAM budget, without changing model outputs.

We make four contributions:

- **A working-set formulation of low-resource MoE serving.** We frame routed expert weights and the KV cache as two memory-demand streams competing for the same limited VRAM. This gives a common vocabulary for expert residency, KV allocation, thrashing, and admission, and separates our low-concurrency setting from high-batch datacenter serving (Section 2).
- **A drop-in routing-aware expert pager.** We implement expert paging through the serving engine's ex-

若干专用系统在这一基线上有所改进，但它们做出了不同的权衡。有些将部分 MoE 计算保留在 CPU 上，例如 ktransformers，它可以很快，但依赖 Intel AMX（KVCache.AI，2024）；而另一些则通过改变权重格式或精度来面向边缘部署，例如 llama.cpp（Gerganov 等，2023）。第三条路线则通过流水线、预取或预测专家传输来将通信隐藏在计算之下（Cao 等，2025；Du 等，2024；Eliseev & Mazur，2023）。最后这条路线在精神上与 WiSP 最接近，但其主要手段是重叠：只有当有足够的计算或批处理来隐藏数据移动时，它才能发挥最佳效果。我们的场景处于光谱的另一端：在低并发的交互式服务中，每一步只激活一小部分被路由的专家，但解码过程暴露出的计算量很少，无法用来隐藏 PCIe 传输。

因此，缺失的一环不是另一种流式加载下一批专家权重的方法，而是一种决定哪些专家应常驻显存、以及该常驻集合如何与KV缓存共享内存的机制。

服务本质上是一个工作集问题。在低资源MoE服务中，稀缺资源是GPU显存和PCIe带宽，而非算力。这使得该问题与经典的工作集问题（Denning，1968；1970）极为相似。GPU显存是小型快速存储，而主机内存是后备存储，模型产生两股内存需求流：由MoE路由器决定的路由专家权重，以及由活跃token决定的KV缓存块。两者在使用时都必须常驻，且都从同一VRAM预算中分配。

在此视角下，良好的服务就是内存管理。系统应将工作负载重复使用的专家保留在GPU显存中，为活跃请求的KV块留出足够空间，并避免接纳超出这两个工作集支撑能力的并发任务。这将专家卸载和KV缓存管理从两个独立的调节旋钮转变为一个统一的分配问题：分配给常驻专家的每一个字节，都是从KV缓存中夺走的字节，反之亦然。这一抽象适用于任何具有路由专家和自回归KV缓存的服务栈，包括MoE大语言模型和新兴的MoE视觉-语言模型。

相同的视角也塑造了系统设计。WiSP 改变的是权重的存放位置以及内存的划分方式，而非计算本身。它保持服务引擎、内核、模型权重和数值精度不变，因此其输出与底层引擎逐字节一致。

WiSP 将该策略实现为即插即用的插件。它复用引擎已有的专家映射间接层，该层将逻辑专家 ID 映射到物理槽位，并将这些槽位变为仅包含常驻专家的小型 GPU 暂存空间。随着工作集的变化，WiSP 更新映射，而不是更改内核、模型或精度。这使得该机制与架构无关，适用于引擎已能服务的任何 MoE 模型。第 3 节详细描述了该机制。

我们的发现。在模型无法完全驻留的情况下，于匹配的 GPU 内存预算下，WiSP 的路由感知分页相比静态卸载可将解码吞吐量提升最高达 2.0×，测量基于物理受限的 24 GiB RTX 3090（PCIe 4.0）。这正是 WiSP 所针对的场景：当模型无法保持常驻时，仅缓存工作负载重复使用的专家所移动的字节数远少于静态层级卸载。一旦有足够的显存使模型常驻，静态常驻会更快，因为 WiSP 的分页间接寻址变得不必要；我们明确报告了这一交叉点。

我们随后测试了自然的下一步思路：利用路由预测来预取专家并隐藏页面错误。在低并发解码中，这带来的收益甚微，甚至可能有害，因为可供隐藏 PCIe 传输的计算量太少，因此解码受带宽限制而非受预测限制；推测性传输大多与需求传输竞争同一链路。因此，路由信息作为内存信号——哪些专家值得保持常驻——比作为预取的延迟信号更有用。

这将论文的核心问题从预取转向了分配。专家权重和KV缓存共享同一VRAM预算：分配给常驻专家的每一个字节都是从KV中拿走的，反之亦然。我们提出了MV-WSA，一种边际价值分配器，通过均衡每字节的延迟收益来划分这一预算，同时强制执行KV准入下限，使调度器仍能接纳所请求的并发量。MV-WSA可作为启动配置器使用，在服务开始前确定划分比例；也可作为在线控制器使用，在请求运行期间调整专家池和KV池的大小。在我们的真实服务分配测试中，配置器是唯一在预填充和解码两方面都接近最优的策略；在线控制器在相同VRAM预算下，进一步将端到端性能较最佳离线划分提升最多 1.19×，且不改变模型输出。

我们做出四项贡献：

- 低资源MoE服务的工作集公式化。我们将路由专家权重和KV缓存视为争夺同一有限VRAM的两条内存需求流。这为专家驻留、KV分配、抖动和准入提供了共同的术语体系，并将我们的低并发场景与高批次数据中心服务区分开来（第2节）。
- 一个即插即用的路由感知专家分页器。我们通过服务引擎现有的专家映射间接层来实现专家分页，

isting expert-map indirection, without changing kernels, model weights, or quantization. On a physically constrained 24 GiB RTX 3090, WiSP improves decode throughput by up to 2.0× over static offload at matched VRAM when the model does not fit (Sections 3, 4).

- **A negative result on prefetching in low-concurrency decode.** We show that routing-based expert prefetching is not the main speed lever in this regime: single-stream decode is PCIe-bandwidth-bound, not prediction-bound, so speculative transfers compete with demand transfers instead of hiding them. The routing signal is more useful for sizing the resident expert set than for reducing latency through prefetching (Section 4).

- **MV-WSA for joint expert–KV allocation.** We introduce a marginal-value allocator that divides VRAM between resident experts and the KV cache under an admission floor. As a startup configurator, it is near-best on both prefill and decode in our real-serving allocation test. As an online controller, it physically resizes both pools during serving and improves end-to-end performance by up to 1.19× over the best offline split at the same VRAM budget, byte-identically (Sections 3, 5).

2 Background: Serving as a Working-Set Problem

2.1 The Working-Set View

A program's *working set* is the set of memory pages it has used recently (Denning, 1968; 1970). If that set stays resident, execution is fast, but if the resident set is too small, the program repeatedly faults on pages it just used and begins to *thrash*. If the system admits more concurrent work than the combined working sets can support, the whole machine thrashes. Low-resource MoE serving has the same shape: each active token needs a routed subset of experts in every MoE layer, and each active request needs its KV blocks. Both must be resident in GPU memory when used. As concurrency, context length, or the routed expert union grows, the two working sets compete for the same limited VRAM.

This view is useful because expert use is sparse, but it is not structureless. A token activates only k of N experts, so keeping every expert resident wastes memory when VRAM is limited. At the same time, a short sequence of decode steps touches much more than a tiny per-token set, so the cache cannot shrink to just the next token's experts. The opportunity is reuse at the workload scale: across bursts and sessions, a workload tends to revisit a stable subset of experts. Caching that routed working set, rather than the whole expert pool, is what makes expert paging viable on a device that cannot keep the full model resident.

The KV cache is the other half of this memory problem. PagedAttention (Kwon et al., 2023) already manages KV as blocks that can be allocated, reused, and spilled. Expert weights, however, are usually handled by a separate mechanism: either kept fully resident or streamed in bulk from the host. The two mechanisms still draw from the same VRAM budget. Every byte used for resident experts is a byte unavailable to KV, and vice versa. Seen this way, low-resource MoE serving is not two independent problems—expert offload and KV management—but one allocation problem over two competing working sets.

2.2 Why the Serving Regime Matters

The best strategy depends on concurrency. In high-batch datacenter serving, a layer may activate many or most experts across the batch. The per-layer working set is then close to the whole expert bank, and there is enough computation to overlap expert transfers with execution. Streaming and prefetching are natural in that setting (Liu et al., 2026; Cao et al., 2025).

WiSP targets the other end of this spectrum: one or a few interactive streams on a GPU that cannot hold the full model. Here each step activates only a small routed subset of experts, and decode exposes little computation behind which to hide PCIe transfers. The useful question is therefore not how to hide the next transfer, but which experts should remain resident across steps and how much VRAM should be left for KV. Section 4 measures this directly.

2.3 Scope and Generality

The expert-paging mechanism applies to any model with routed experts that the serving stack presents as an expert bank. The full working-set allocation problem applies when the model also has an autoregressive KV cache, because resident experts and KV blocks compete for the same memory budget. Models without a KV cache, such as diffusion MoEs or pure recurrent SSM–MoE variants, can still use expert paging to reduce resident memory. They do not exercise the full expert–KV allocation problem, so we treat them as mechanism-generalization cases rather than headline throughput targets. Table 1 summarizes this distinction.

Where WiSP sits among existing systems. Most prior MoE-offloading work targets autoregressive MoE LLMs. The main axis separating these systems is the serving regime. High-batch systems can stream or prefetch experts because they have enough computation to overlap transfers. WiSP targets low-concurrency serving, where overlap is limited and reuse-based caching matters more. It changes residency and allocation policy, not the model computation, so the engine's outputs remain byte-identical.

The closest concurrent system is FluxMoE (Liu et al., 2026).

无需更改内核、模型权重或量化。在物理受限的 24GiB RTX 3090 上，当模型无法完全容纳时，WiSP 在匹配显存条件下将解码吞吐量较静态卸载提升至多 2.0×（第 3、4 节）。

• 在低并发解码中预取的一个负面结果。我们表明，基于路由的专家预取在此场景下并不是主要的速度杠杆：单流解码受限于PCIe带宽，而非受限于预测，因此推测性传输会与需求传输竞争，而不是将其隐藏。路由信号在确定常驻专家集大小方面比通过预取降低延迟更有用（第4节）。

• 用于专家–KV联合分配的MV-WSA。我们引入了一种边际价值分配器，在准入下限约束下将VRAM分配给常驻专家和KV缓存。作为启动配置器，在我们的真实服务分配测试中，它在预填充和解码两方面都接近最优。作为在线控制器，它在服务期间物理调整两个池的大小，在相同VRAM预算下，相比最佳离线划分，端到端性能提升最高可达 1.19×，且字节完全相同（第3、5节）。

2 背景：作为工作集问题的服务

2.1 工作集视角

程序的工作集是它最近使用过的内存页集合（Denning，1968；1970）。如果该集合保持驻留，执行就很快；但如果驻留集太小，程序就会反复对其刚刚使用过的页面发生缺页，并开始抖动。如果系统接纳的并发工作超过合并工作集所能支持的量，整个机器就会抖动。低资源 MoE 服务具有相同的形态：每个活跃 token 都需要每个 MoE 层中被路由到的专家子集，而每个活跃请求都需要其 KV 块。两者在使用时都必须驻留在 GPU 内存中。随着并发量、上下文长度或路由专家并集的增长，这两个工作集会争夺同一有限的 VRAM。

这一视角之所以有用，是因为专家的使用虽然是稀疏的，但并非无结构。一个 token 只激活 N 个专家中的 k 个，因此在显存有限时，让每个专家都常驻会浪费内存。与此同时，一段短的解码步骤序列所触及的专家远不止每 token 的极小集合，因此缓存不能缩小到仅保留下一个 token 的专家。机会在于工作负载尺度上的复用：跨突发和会话，工作负载往往会重新访问一个稳定的专家子集。缓存这一路由工作集，而非整个专家池，正是使专家分页在无法将完整模型常驻的设备上可行的关键。

KV 缓存是这一内存问题的另一半。PagedAttention（Kwon 等，2023）已经将 KV 作为可分配、可复用、可溢出的块来管理。然而，专家权重通常由另一套机制处理：要么完全常驻，要么从主机批量流式加载。这两种机制仍然从同一显存预算中取用。用于常驻专家的每一个字节，都是 KV 无法使用的字节，反之亦然。由此看来，低资源 MoE 服务并非两个独立问题——专家卸载与 KV 管理——而是两个相互竞争的工作集之上的同一个分配问题。

2.2 为什么服务机制很重要

最佳策略取决于并发量。在高批量数据中心服务中，一个层可能会在整个批次中激活许多或大多数专家。此时每层工作集接近于整个专家库，并且有足够的计算来将专家传输与执行重叠起来。流式传输和预取在这种场景下是自然的选择（Liu et al., 2026; Cao et al., 2025）。

WiSP 针对的是这一频谱的另一端：在无法容纳完整模型的 GPU 上运行一个或少数几个交互式流。此时每一步仅激活一小部分被路由的专家，而解码阶段几乎没有计算量可用于掩盖 PCIe 传输。因此，有用的问题不在于如何隐藏下一次传输，而在于哪些专家应在各步骤间保持驻留，以及应为 KV 留出多少显存。第 4 节直接对此进行了测量。

2.3 范围与通用性

专家分页机制适用于任何具有路由专家且服务栈将其呈现为专家库的模型。完整工作集分配问题在模型同时具有自回归 KV 缓存时出现，因为驻留专家和 KV 块竞争同一内存预算。没有 KV 缓存的模型，例如扩散 MoE 或纯循环 SSM–MoE 变体，仍可使用专家分页来减少驻留内存。它们不涉及完整的专家–KV 分配问题，因此我们将其视为机制泛化案例，而非主要吞吐量目标。表 1 总结了这一区别。

WiSP 在现有系统中所处的位置。以往大多数 MoE 卸载工作都针对自回归 MoE 大语言模型。区分这些系统的主要维度是服务模式。高批量系统能够流式传输或预取专家，因为它们有足够的计算来重叠传输。WiSP 面向低并发服务，在这种情况下重叠有限，基于复用的缓存更为重要。它改变的是驻留和分配策略，而非模型计算，因此引擎的输出保持逐字节完全相同。

最接近的并发系统是 FluxMoE（Liu 等，2026）。

Table 1. Where the working-set view applies. *Expert stream* = routed experts that can be paged; *KV stream* = an autoregressive key–value cache that co-contends for memory. WiSP's mechanism needs the former; its full theory needs both.

Architecture	MoE variant	Representative	Expert stream	KV stream	WiSP coverage
Dense LLM	× (dense)	Llama-3, Qwen2.5	×	✓	n/a
SSM (pure)	✓	BlackMamba (Anthony et al., 2024)	✓	× (recurrent state)	mechanism only
MoE LLM	✓	Mixtral, Qwen3-MoE, DeepSeek-V3	✓	✓	**full (primary)**
MoE VLM	✓	DeepSeek-VL2, Kimi-VL, Aria	✓	✓	**full (breadth)**
Hybrid SSM–MoE	✓	Jamba (Lieber et al., 2024)	✓	✓(attention)	**full**
MoE diffusion LLM	✓	LLaDA-MoE (Zhu et al., 2025)	✓	× (bidirectional)	mechanism only
MoE image diffusion	✓	DiT-MoE (Fei et al., 2024)	✓	× (latent denoise)	mechanism only

Like WiSP, it trades expert residency against KV capacity, but it uses a stream-and-evict policy tuned for high-batch overlap. WiSP instead uses cache-and-reuse for the regime where there is little compute to hide transfer latency. Beyond autoregressive LLMs, MoE VLMs, hybrid models, and diffusion MoEs use different serving stacks; WiSP attaches to them either as the same serving stack plug-in or through a small wrapper. Appendix D gives the full design-space and serving-stack comparisons (Tables 5, 6).

3 WISP

WiSP separates low-resource MoE serving into two decisions. First, a *routing-aware expert pager* (§3.2) decides which experts are resident in GPU memory. It uses the serving engine's existing `expert_map` indirection to map routed expert ids onto a small GPU scratch space, so the kernel, weights, precision, and outputs are unchanged. Second, *MV-WSA* (Marginal-Value Working-Set Allocation, §3.3–§3.5) decides how much of the available VRAM should go to that expert scratch versus the KV cache. The pager manages residency *within* the expert pool; MV-WSA moves the boundary *between* the expert and KV pools. We use MV-WSA in two modes: offline, as a startup configurator that fixes the split before serving, and online, as a controller that resizes both pools while requests are running. Figure 1 summarizes the design.

3.1 Setting and Notation

The model has L MoE layers with N experts each, and the router selects $k \ll N$ experts per token per layer. At step t, layer ℓ selects $R_\ell^{(t)} \subseteq \{1, \dots, N\}$, and every expert $e \in R_\ell^{(t)}$ must be resident when that layer executes. After fixed GPU costs are reserved—non-expert weights, activations, CUDA context, and other runtime state—the device has a pageable budget B shared by resident experts and the KV cache:

$$B = B_{\text{exp}} + B_{\text{kv}}, \qquad f \triangleq B_{\text{exp}}/B \in (0,1). \quad (1)$$

The split f is MV-WSA's control variable. If one expert occupies M_e bytes, then increasing the per-layer resident cap by one expert costs $a \triangleq LM_e$ bytes across all MoE layers, so the expert budget supports $C = B_{\text{exp}}/a$ resident experts per layer. The KV budget supports $\kappa = B_{\text{kv}}/b$ KV blocks, where b is the GPU memory per KV block summed over all layers. At a fixed budget B, WiSP's allocation problem is to choose where this split should land.

3.2 The Expert Pager

The allocator can only trade expert memory against KV memory if the expert side is itself pageable. WiSP provides this substrate with the engine's existing expert-map indirection, without changing the fused-MoE kernel, model weights, or precision.

Expert-map indirection. Modern fused-MoE kernels accept an `expert_map` π that maps logical expert ids to physical slots. WiSP makes those physical slots a small GPU scratch space with capacity C per layer. The full expert weights remain in host memory; at each layer, WiSP ensures that the routed experts are present in scratch, updates π, and dispatches the unmodified kernel. A page fault occurs when a routed expert is not currently resident, in which case WiSP copies it from the host master and evicts an older resident expert using LRU. All dynamic behavior is in the map and the scratch contents, not in the computation.

Per-step protocol. Algorithm 1 shows the per-layer decode step. Given the routed expert set R, WiSP copies only the missing experts, assigns them scratch slots, refreshes the LRU state for the experts used by this step, and calls the stock fused-MoE kernel with the updated expert map. The key invariant is exactness. When the kernel runs, every routed expert id maps to a scratch slot containing the same bytes the full expert tensor would have supplied. The kernel is unchanged, so WiSP's output matches the unpaged engine up to the engine's own floating-point reproducibility. For large prefill batches whose expert union exceeds C, WiSP groups tokens so that each group fits the scratch and reassembles the outputs in token order; this path is an implementation detail and does not trigger in our single-stream decode measurements.

表1. 工作集视角的适用范围。专家流 = 可被分页的路由专家；KV流 = 与专家流争用内存的自回归键值缓存。WiSP的机制需要前者；其完整理论需要两者。

架构	MoE变体	代表	专家流	KV 流	WiSP 覆盖范围
稠密 LLM	×（稠密）	Llama-3、Qwen2.5	×	✓	n/a
SSM（纯）	✓	BlackMamba（Anthony 等人，2024）	✓	×（循环状态）	仅机制
MoE LLM	✓	Mixtral、Qwen3-MoE、DeepSeek-V3	✓	✓	完整（主要）
MoE VLM	✓	DeepSeek-VL2、Kimi-VL、Aria	✓	✓	完整（广度）
混合 SSM–MoE	✓	Jamba（Lieber 等，2024）	✓	✓（注意力）	**full**
MoE扩散大语言模型	✓	LLaDA-MoE（Zhu等，2025）	✓	×（双向）	仅机制
MoE图像扩散	✓	DiT-MoE（Fei等，2024）	✓	×（潜在去噪）	仅机制

与 WiSP 类似，它在专家驻留与 KV 容量之间进行权衡，但采用了针对高批量重叠调优的流式逐出策略。WiSP 则采用缓存复用策略，适用于计算资源很少、无法隐藏传输延迟的场景。除自回归 LLM 之外，MoE VLM、混合模型和扩散 MoE 使用不同的服务栈；WiSP 既可以作为同一服务栈插件接入它们，也可以通过一个小型封装器接入。附录 D 给出了完整的设计空间和服务栈比较（表 5、表 6）。

3 WISP

WiSP 将低资源 MoE 服务拆分为两个决策。首先，路由感知专家分页器（§3.2）决定哪些专家常驻 GPU 内存。它利用服务引擎现有的专家映射间接机制，将路由到的专家 ID 映射到一小块 GPU 暂存空间，因此内核、权重、精度和输出均保持不变。其次，MV-WSA（边际价值工作集分配，§3.3–§3.5）决定可用 VRAM 中有多少应分配给该专家暂存空间，多少分配给 KV 缓存。分页器管理专家池内的常驻；MV-WSA 则移动专家池与 KV 池之间的边界。我们以两种模式使用 MV-WSA：离线模式，作为启动配置器，在服务开始前固定划分；在线模式，作为控制器，在请求运行期间动态调整两个池的大小。图 1 总结了该设计。

3.1 设定与符号

模型有 L 个 MoE 层，每层有 N 个专家，路由器为每个 token 在每层选择 $k \ll N$ 个专家。在第 t 步，第 ℓ 层选择 $R_\ell^{(t)} \subseteq \{1, \dots, N\}$，并且当该层执行时，每个专家 $e \in R_\ell^{(t)}$ 必须常驻。在预留固定 GPU 开销——非专家权重、激活值、CUDA 上下文及其他运行时状态——之后，设备有一个可分页预算 B，由常驻专家和 KV 缓存共享：

$$B = B_{\text{exp}} + B_{\text{kv}}, \qquad f \triangleq B_{\text{exp}}/B \in (0,1). \quad (1)$$

分割 f 是MV-WSA的控制变量。如果一个专家占用 M_e 字节，那么将每层常驻上限增加一个专家，在所有MoE层上总计需要 $a \triangleq LM_e$ 字节的开销

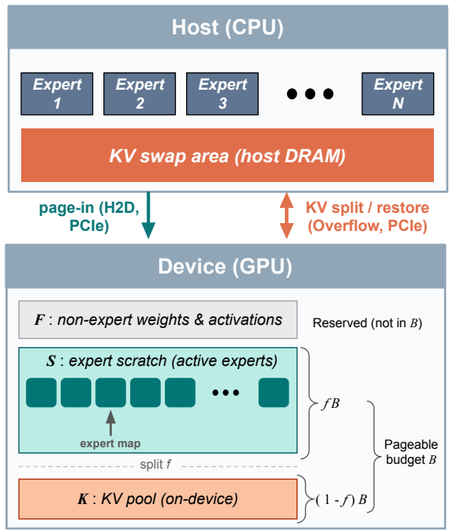
因此专家预算支持每层 $C = B_{\text{exp}}/a$ 个常驻专家。KV预算支持 $\kappa = B_{\text{kv}}/b$个KV块，其中 b 是所有层上每个KV块的GPU内存之和。在固定预算 B下，WiSP的分配问题就是选择这个分割应该落在哪里。

3.2 专家分页器

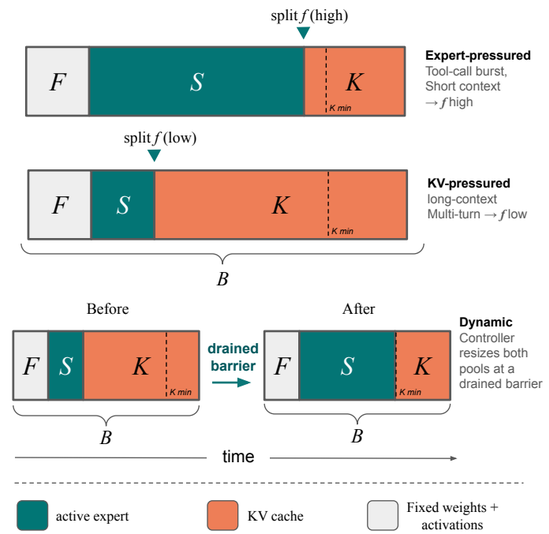
分配器只有在专家侧本身可分页时，才能在专家内存与 KV 内存之间进行权衡。WiSP 利用引擎现有的专家映射间接层提供了这一基础，而不改变融合 MoE 内核、模型权重或精度。

专家映射间接层。现代融合 MoE 内核接受一个专家映射 π，它将逻辑专家 id 映射到物理槽位。WiSP 将这些物理槽位变成一个每层容量为 C 的小型 GPU 暂存空间。完整的专家权重保留在主机内存中；在每一层，WiSP 确保被路由的专家存在于暂存空间中，更新 π，并调度未经修改的内核。当被路由的专家当前未驻留时，就会发生缺页，在这种情况下，WiSP 从主机主副本复制它，并使用 LRU 驱逐一个较旧的常驻专家。所有动态行为都在映射和暂存空间内容中，而不在计算中。

逐步协议。算法 1 展示了逐层解码步骤。给定路由专家集合 R，WiSP 仅复制缺失的专家，为其分配临时槽位，刷新本步骤所用专家的 LRU 状态，并使用更新后的专家映射调用原生的融合 MoE 内核。关键不变量是精确性。当内核运行时，每个路由专家 id 都映射到一个临时槽位，其中包含与完整专家张量本应提供的相同字节。内核未作改动，因此 WiSP 的输出与未分页引擎一致，差异仅取决于引擎自身的浮点可复现性。对于专家并集超过 C 的大型预填充批次，WiSP 将令牌分组，使每组都适配临时空间，并按令牌顺序重组输出；此路径属于实现细节，在我们单流解码测量中不会触发。



(a) System Layout



(b) Runtime regimes: MV-WSA moves the split

Figure 1. WiSP at a glance. **(a) System layout.** The full expert weights stay pinned on the host; WiSP *pages in* only the routed experts to a small GPU scratch S over PCIe and runs the unmodified fused-MoE kernel through the `expert_map` π, so output is byte-identical. The pageable budget $B = B_{\text{exp}} + B_{\text{kv}}$—the expert scratch S plus the KV pool K, *excluding* the fixed non-expert slab F—is divided by the split f; KV that overflows the on-device pool spills to host. **(b) Runtime regimes.** The optimal split moves with the workload—expert-heavy for tool-call bursts, KV-heavy for long contexts—and MV-WSA tracks it, in the dynamic case by physically resizing both pools at a drained barrier, byte-identically.

Algorithm 1 WISPLAYER.STEP(R) at one MoE layer

Input: needed experts R; cap C; maps π, π^{-1}; LRU ticks τ; clock c; CPU master W^{cpu}; GPU scratch S; device map π_{dev}
$M \leftarrow \{e \in R : \pi(e) = \perp\}$ ▷ faults
$U \leftarrow \{s : \pi^{-1}(s) \notin R\}$ in increasing τ ▷ LRU victims
ASSIGN each $e \in M$ a victim slot s_e from U
for each $e \in M$ with slot s_e **do**
  $S[s_e] \leftarrow$ COPY_ASYNC($W^{\text{cpu}}[e]$)
  $\pi(e) \leftarrow s_e$; $\pi^{-1}(s_e) \leftarrow e$; $\pi_{\text{dev}}[e] \leftarrow s_e$
end for
$c \leftarrow c + 1$; **for** $e \in R$ **do** $\tau[\pi(e)] \leftarrow c$
return FUSEDMOE(hidden, W=S, `expert_map`=π_{dev})

Reuse-distance miss curves. WiSP uses LRU because the low-concurrency decode regime is reuse-limited rather than overlap-limited: each step has too little computation to hide a PCIe transfer, so the useful policy is to keep the experts a workload comes back to. LRU also gives a simple way to estimate the value of more memory. From a reference trace, the reuse-distance histogram determines the miss curve $m(C)$: how many references would miss at resident cap C. We use this miss-curve view for the expert stream, and use the analogous block-level view for KV, with the admission floor below capturing the scheduler cliff that a smooth miss curve misses.

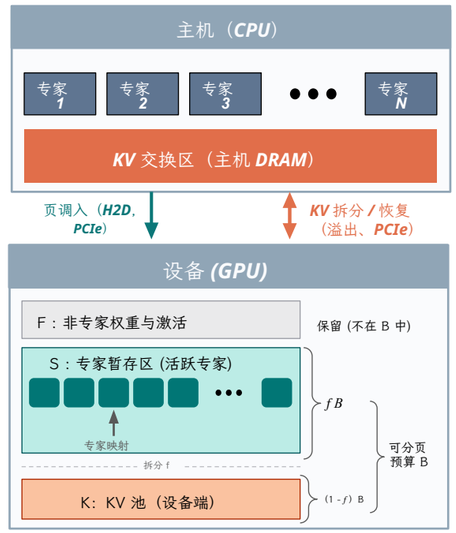
3.3 The Expert↔KV Split

A split f gives $C = fB/a$ expert slots per layer and $\kappa = (1-f)B/b$ KV blocks. The allocator asks a simple question: if we move one byte from KV to experts, do we remove more latency than if we leave that byte in KV?

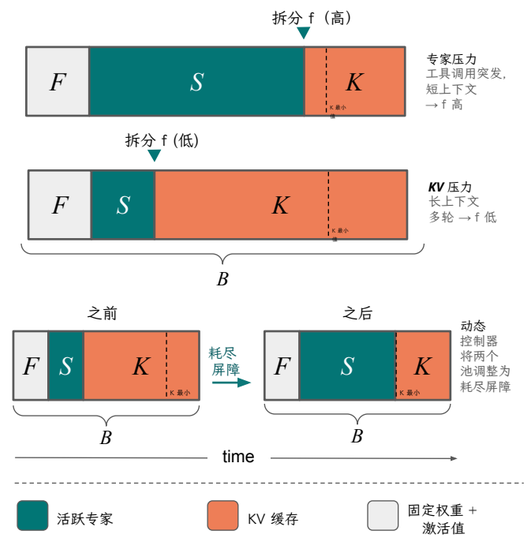
Let $m_{\text{exp}}(C)$ be the expert miss count at cap C, and let $m_{\text{kv}}(\kappa)$ be the KV miss or recompute cost induced by a KV pool of κ blocks. Expert misses and KV misses have different costs. An expert miss transfers weights over PCIe, with cost $c_{\text{exp}} = M_e t_{\text{byte}}$. A KV miss costs recomputation, with cost $c_{\text{kv}} = \beta b_{\text{tok}}$. The latency model at split f is

$$T(f) \;=\; c_{\text{exp}}\, m_{\text{exp}}(C) \;+\; c_{\text{kv}}\, m_{\text{kv}}(\kappa). \qquad (2)$$

The best split depends on the workload. Short tool-call bursts have little KV pressure and benefit from keeping more reused experts resident, so the optimum shifts expert-heavy. Long-context or multi-turn workloads put more pressure on KV, so the optimum shifts back toward KV. At an interior



(a) 系统布局



(b) 运行时状态：MV-WSA 移动分割点

图 1. WiSP 概览。(a) 系统布局。完整的专家权重固定驻留在主机上；WiSP 仅通过 PCIe 将路由到的专家分页到小型 GPU 暂存区 S，并通过专家映射 π 运行未经修改的融合 MoE 内核，因此输出字节级一致。可分页预算 $B = B_{\text{exp}} + B_{\text{kv}}$——专家暂存区 S 加上 KV 池 K，不包括固定的非专家区块 F——按拆分 f 进行划分；超出设备端池的 KV 会溢出到主机。(b) 运行时模式。最优拆分随工作负载变化——在工具调用突发时偏向专家，在长上下文时偏向 KV——MV-WSAtracks 会跟踪它，在动态情况下通过在排空屏障处物理调整两个池的大小，实现字节级一致。

算法 1 WISPLAYER.STEP(R) 在单个 MoE 层

输入：所需专家 R；容量 C；映射 π, π^{-1}；LRU 计时 τ；时钟 c；CPU 主存 W^{cpu}；GPU 暂存 S；设备映射 π_{dev}
$M \leftarrow \{e \in R : \pi(e) = \perp\}$ ▷ 缺页
$U \leftarrow \{s : \pi^{-1}(s) \notin R\}$按递增 τ ▷LRU 淘汰项
为每个 $e \in M$ 分配一个淘汰槽位 s_e，来自 U
对于每个带有槽位 s_e 的 $e \in M$ 执行
  $S[s_e] \leftarrow$ 异步复制($W^{\text{cpu}}[e]$)
  $\pi(e) \leftarrow s_e$；$\pi^{-1}(s_e) \leftarrow e$；$\pi_{\text{dev}}[e] \leftarrow s_e$
结束循环
$c \leftarrow c + 1$；对于 $e \in R$ 执行 $\tau[\pi(e)] \leftarrow c$
return FUSEDMOE(hidden W=S, expert map=π_{dev})

重用距离缺失曲线。WiSP 使用 LRU，因为低并发解码场景受限于重用而非重叠：每一步的计算量太小，无法隐藏 PCIe 传输，因此有用的策略是保留工作负载会再次访问的专家。LRU 还提供了一种简单的方法来估计更多内存的价值。根据参考轨迹，重用距离直方图确定缺失曲线 $m(C)$：在常驻容量 C 下会有多少次引用发生缺失。我们将这种缺失曲线视角用于专家流，并将类似的块级视角用于 KV，其中准入下限低于捕获调度器悬崖，而平滑的未命中曲线会错过这一点。

3.3 Expert↔KV 拆分

一个拆分 f 为每层提供 $C = fB/a$ 个expert槽位和 $\kappa = (1-f)B/b$ 个KV块。分配器会问一个简单的问题：如果我们把一个字节从KV移到experts，是否比将该字节留在KV中消除更多延迟？

设 $m_{\text{exp}}(C)$ 为容量 C下的expert未命中计数，设$m_{\text{kv}}(\kappa)$ 为由 κ 个块的KV池引起的KV未命中或重计算成本。Expert未命中和KV未命中具有不同的成本。Expert未命中通过PCIe传输权重，成本为 $c_{\text{exp}} = M_e t_{\text{byte}}$。KV未命中导致重计算，成本为 $c_{\text{kv}} = \beta b_{\text{tok}}$。拆分 f 处的延迟模型为

$$T(f) \;=\; c_{\text{exp}}\, m_{\text{exp}}(C) \;+\; c_{\text{kv}}\, m_{\text{kv}}(\kappa). \qquad (2)$$

最佳划分取决于工作负载。短工具调用突发对KV压力很小，并且受益于保留更多常驻复用的专家，因此最优配置偏向专家侧。长上下文或多轮工作负载对KV施加更大压力，因此最优配置又偏向KV侧。在一个内点

optimum, the split is equimarginal: one more byte should buy the same latency reduction in either pool,

$$c_{\text{exp}}\, m'_{\text{exp}}(C)/a \;=\; c_{\text{kv}}\, m'_{\text{kv}}(\kappa)/b. \tag{3}$$

MV-WSA follows this principle: move bytes toward whichever pool currently removes more latency per byte.

The admission floor. The smooth cost in Eq. (2) misses one hard constraint. A continuous-batching scheduler must have enough KV capacity to admit the active requests. If the GPU KV pool is too small, the scheduler may preempt and re-prefill repeatedly, or the engine may refuse to initialize. This is a cliff, not a gradual miss penalty. We therefore enforce

$$\kappa \;\geq\; \kappa_{\min} \;=\; W \cdot \lceil \ell_{\text{ctx}}/b_{\text{tok}} \rceil, \tag{4}$$

where W is the served concurrency and ℓ_{ctx} is the per-session context length. The floor is slack when KV is the pressured pool. It becomes binding when the unconstrained optimum pushes the split toward experts: $\kappa_{\min}$ is what keeps an expert-heavy split admissible. This is the difference between MV-WSA and a KV-blind expert-proportional split, which we measure as a startup failure in Section 5.3.

3.4 MV-WSA: Configurator and Online Controller

MV-WSA does not need the full miss curves in advance. It builds them from decayed reuse-distance histograms while the workload runs, then moves the split toward the equimarginal point in Eq. (3). A small hysteresis band prevents the split from chasing noise. We use the same allocation principle in two modes.

In *offline* mode, MV-WSA runs on a short representative trace and returns a fixed split $\hat{f}$ used at startup. Given a budget B, it emits the two engine knobs that realize this split: the per-layer expert cap and the GPU KV pool, subject to the admission floor,

$$C = \min\big(\lfloor \hat{f}B/a \rfloor,\, \lfloor (B - \kappa_{\min} b)/a \rfloor\big), \qquad B_{\text{kv}} = B - Ca. \tag{5}$$

All other memory settings are held fixed, so comparisons are iso-VRAM: the arms differ only in where the same bytes go.

In *online* mode, MV-WSA updates the split while serving. A single startup split is only the best fixed compromise over a trace, but an agentic session can move between regimes: long-context turns may need KV, while short tool-call bursts may leave KV underused and benefit from more resident experts.

3.5 Dynamic Dual-Resize

The live controller implements the online case using a simple observation from the engine: once the KV pool covers the current working set, stays above $\kappa_{\min}$, and has a small amount of headroom, extra KV blocks have near-zero marginal value. The controller therefore measures the peak KV occupancy at a drained barrier, keeps KV just above that peak, and gives the remaining bytes to the expert scratch. In this regime, the equimarginal rule becomes simple: once KV has enough room, the next byte is worth more on the expert side. Appendix B gives the full control law.

Applying the new split inside a running engine requires two resize operations: resizing the per-layer expert scratch and resizing the vLLM KV-block pool. Both happen only at a drained barrier, when no request owns a block being moved. The controller frees the shrinking side before growing the other, so the transient footprint never exceeds the fixed budget B. The result is still an iso-VRAM comparison, in which the controller only moves bytes from one pool to the other. Because it moves stored tensors rather than changing computation, greedy outputs are preserved. If a resize fails, the engine keeps the previous valid split.

3.6 Implementation

WiSP is implemented as a drop-in plug-in over an unmodified vLLM. Expert masters live in host memory, and each MoE layer owns a GPU scratch whose size is set by the expert cap C. The static configurator realizes a split by setting the expert cap and the on-device KV pool; the online controller adds resize operations for both pools and applies them only at drained barriers. Appendix C gives the engineering details.

4 The Central Finding: Prediction Is a Memory Lever, Not a Speed Lever

The working-set view suggests a natural optimization: if we can predict which experts a token will need, we should prefetch them and hide page faults. This is not a strawman. Routing is partly predictable because later-layer routing depends on hidden states produced by earlier experts, and our co-activation predictor reaches nontrivial next-expert precision (Appendix E). The question is whether that prediction turns into decode speed.

In the low-concurrency regime, it does not. Figure 2 tests speculative expert prefetching on Qwen3-30B-A3B on a real 24 GiB RTX 3090 (PCIe 4.0). Turning on prefetch lowers decode throughput by 38–60% at constrained caps and raises time-to-first-token by 1.6–1.9×. A second test compares a topic-coherent session with a deliberately diverse one; despite having more routing locality, the coherent session reaches no higher steady-state throughput (6.47 vs. 6.82 tok/s). There is no warm-up benefit to harvest.

The reason is bandwidth. At batch-1 decode, each layer

最优处，划分是等边际的：多一个字节在任一池中都应带来相同的延迟降低，

$$c_{\text{exp}}\, m'_{\text{exp}}(C)/a \;=\; c_{\text{kv}}\, m'_{\text{kv}}(\kappa)/b. \tag{3}$$

MV-WSA遵循这一原则：将字节移向当前每字节能消除更多延迟的池。

准入下限。式(2)中的平滑成本遗漏了一个硬约束。连续批处理调度器必须拥有足够的KV容量来接纳活跃请求。如果GPU KV池太小，调度器可能会反复抢占和重新预填充，或者引擎可能拒绝初始化。这是一个悬崖，而非渐进的未命中惩罚。因此我们强制

$$\kappa \;\geq\; \kappa_{\min} \;=\; W \cdot \lceil \ell_{\text{ctx}}/b_{\text{tok}} \rceil, \tag{4}$$

其中 W 是服务并发数，ℓ_{ctx} 是每会话上下文长度。当KV是受压池时，该下限是松弛的。当无约束最优解将分割推向专家时，它变得具有约束力：$\kappa_{\min}$ 是保持专家密集型分割可准入的关键。这就是MV-WSA与KV盲的专家比例分割之间的区别，我们在第5.3节中将其测量为启动失败。

3.4 MV-WSA：配置器与在线控制器

MV-WSA 无需预先获取完整的缺失曲线。它会在工作负载运行期间，基于衰减的重用距离直方图构建这些曲线，然后将切分点移向式 (3) 中的等边际点。一个小的迟滞带可防止切分点追逐噪声。我们在两种模式下使用相同的分配原则。

在离线模式下，MV-WSA 在一条简短的代表性 ˆ 轨迹上运行，并返回一个在启动时使用的固定切分 f。给定预算 B，它输出实现该切分的两个引擎旋钮：逐层专家上限和 GPU KV 池，并受准入下限约束，

$$C = \min\big(\lfloor \hat{f}B/a \rfloor,\, \lfloor (B - \kappa_{\min} b)/a \rfloor\big), \qquad B_{\text{kv}} = B - Ca. \tag{5}$$

所有其他内存设置均保持固定，因此比较是等 VRAM 的：各实验组仅在相同字节的去向上有所不同。

在在线模式下，MV-WSA 在服务过程中更新切分。单一启动切分只是轨迹上的最佳固定折衷，但智能体会话可能在多种模式之间切换：长上下文轮次可能需要 KV，而短工具调用突发可能使 KV 利用不足，并从更多常驻专家中获益。

3.5 动态双重调整大小

在线控制器利用引擎中的一个简单观察来实现在线场景：一旦 KV 池覆盖当前工作集，保持在 $\kappa_{\min}$ 以上，并且有少量余量，额外的 KV 块边际价值几乎为零。因此，控制器在排空屏障处测量峰值 KV 占用，将 keepKV 保持在该峰值略高之处，并将剩余字节分配给专家暂存区。在这种情形下，等边际规则变得简单：一旦 KV 有足够空间，下一个字节在专家侧更有价值。附录 B 给出了完整的控制律。

在运行中的引擎内应用新的分割需要两次调整大小操作：调整每层专家暂存区的大小，以及调整 vLLM KV 块池的大小。两者都只在排空屏障处发生，此时没有任何请求拥有正在被移动的块。控制器先释放收缩的一侧，再扩展另一侧，因此瞬态占用永远不会超过固定预算 B。结果仍然是等显存比较，其中控制器只是将字节从一个池移动到另一个池。由于它移动的是存储的张量而非改变计算，贪婪输出得以保留。如果调整大小失败，引擎会保留先前的有效分割。

3.6 实现

WiSP 是作为未修改 vLLM 上的即插即用插件实现的。专家主副本驻留在主机内存中，每个 MoE 层拥有一个 GPU 暂存区，其大小由专家上限 C 设定。静态配置器通过设置专家上限和设备上的 KV 池来实现分割；在线控制器为两个池添加调整大小操作，并仅在排空屏障处应用它们。附录 C 给出了工程细节。

4 核心发现：预测是内存杠杆，而非速度杠杆

工作集视角暗示了一种自然的优化思路：如果我们能预测一个 token 需要哪些专家，就应该预取其所需专家并隐藏页错误。这并非稻草人论证。路由在一定程度上是可预测的，因为后层的路由依赖于前层专家所产生的隐藏状态，而且我们的共激活预测器达到了非平凡的下一专家精确度（附录 E）。问题在于，这种预测能否转化为解码速度。

在低并发场景下，它并不能。图 2 在一台真实的 24GiB RTX 3090（PCIe 4.0）上测试了 Qwen3-30B-A3B 上的推测性专家预取。开启预取后，在受限上限下解码吞吐量降低了 38–60%，并将首 token 时间提高了 1.6–1.9× 倍。第二项测试比较了主题一致的会话与刻意多样化的会话；尽管主题一致的会话具有更高的路由局部性，其稳态吞吐量并未更高（6.47 vs. 6.82 tok/s）。没有任何预热收益可供获取。

原因在于带宽。在 batch-1 解码时，每一层

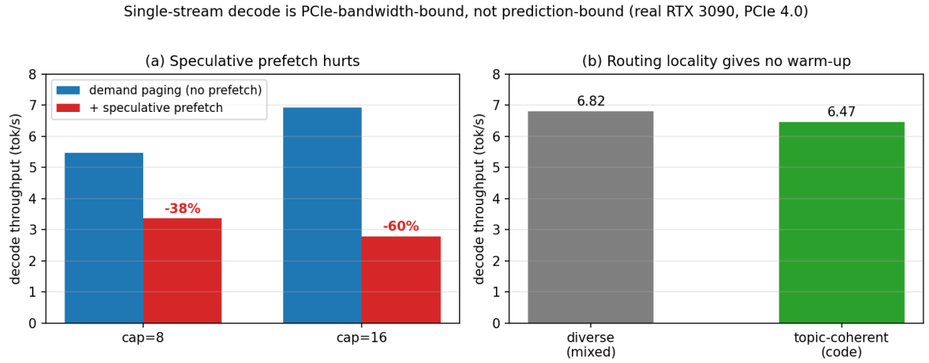


Figure 2. Prediction does not buy decode speed at single-stream, measured on a real 24 GiB RTX 3090 (PCIe 4.0). (a) Turning on speculative prefetch with the online co-activation predictor *lowers* decode throughput by 38–60% at constrained caps (cap 8: 5.47 → 3.37; cap 16: 6.94 → 2.79 tok/s) and raises time-to-first-token by 1.6–1.9× (Qwen3-30B-A3B). (b) A topic-coherent (code) session reaches no higher steady-state throughput than a deliberately diverse one (6.47 vs. 6.82 tok/s), so there is no routing-locality warm-up to harvest. The binding constraint is PCIe bandwidth, not prediction quality.

computes a single token, so the MoE GEMM is too small to hide a side-stream PCIe transfer. Speculative copies therefore compete with demand copies for the same link, and at tight caps they can evict experts that the next step still needs. The measured traffic makes the limit concrete: on Qwen3-30B-A3B, only 44–48% of each layer's top-8 experts carry over between adjacent decode steps. Even a perfect last-step cache must still stream about 1.8 GiB of expert weights per token. On our RTX 3090, with about 25 GB/s host-to-device bandwidth, that mandatory traffic alone caps decode at roughly 13–16 tok/s. The pipe is already full; prediction can reshuffle traffic, but it cannot widen the pipe.

This result changes how we use routing information. In this regime, routing is not a latency signal for prefetching; it is a memory signal for deciding how large the resident expert set should be. Once expert paging frees part of the model from GPU residency, the next question is how to spend that freed memory: keep more experts resident, or give the bytes to the KV cache? That allocation problem is what MV-WSA solves (§3.3–§3.5) and what we evaluate in §5.3.

5 Evaluation

The evaluation follows the paper's argument. First, WiSP should not change the model's computation, so we check output preservation (Section 5.1). Second, if expert residency is a working-set problem, routing-aware paging should move less data than static layer-level offload at the same GPU budget; we test this on a physically constrained 24 GiB RTX 3090 (Section 5.2). Third, because expert memory and KV memory share one store, we evaluate whether MV-WSA chooses a better split than fixed or memory-blind policies, both as a startup configurator and as a live resize controller (Section 5.3).

Experimental setup and scope. The headline iso-VRAM throughput results for Qwen3-30B-A3B and Kimi-VL-A3B (Figure 3, Table 2) are measured on a physically constrained consumer GPU: a single 24 GiB NVIDIA RTX 3090 on a real PCIe 4.0 x16 link (measured host-to-device bandwidth ≈25 GB/s), vLLM 0.11.2, eager mode, temperature 0. Both models exceed 24 GiB in BF16 (Qwen3 ≈57 GiB, Kimi-VL ≈31 GiB), so paging is forced by the hardware rather than emulated. The H100 results serve different purposes: MiniMax-M2 and Jamba are capability checks on models that exceed one 94 GiB H100, while the OLMoE MV-WSA startup-configurator test isolates the allocation policy under a controlled 8 GiB budget. The diffusion experiments use a standalone PyTorch pager, so we report correctness and resident-memory behavior there, not serving latency.

5.1 Paging Is Output-Preserving

At temperature 0, WiSP matches the unmodified engine for every cap we test, and this is the expected consequence of property P1: the fused-MoE kernel is unchanged, and every routed expert is read from byte-equal weights. We also check the mechanism on diffusion models, where the comparison is stricter because we can inspect the whole generation trajectory: DiT-MoE latents match with maximum absolute difference zero across the cap sweep, and LLaDA-MoE completions are token-identical (Section 5.4).

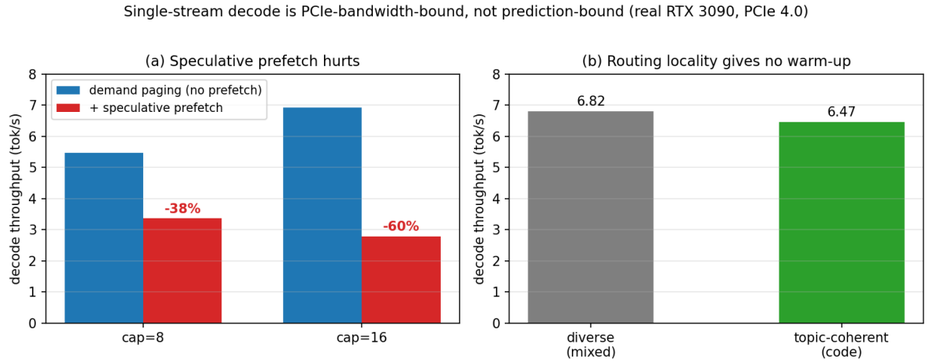


图2. 在真实的24 GiB RTX 3090（PCIe 4.0）上测量，预测在单流下并不能换来解码速度。(a) 在受限上限下，开启在线共激活预测器的推测性预取使解码吞吐量降低38–60%（上限8： 5.47 → 3.37；上限16： 6.94 → 2.79 tok/s），并将首令牌时间提高1.6–1.9× （Qwen3-30B-A3B）。(b) 主题连贯（代码）会话的稳态吞吐量并不高于刻意多样化的会话（6.47 vs. 6.82 tok/s），因此没有路由局部性预热可供利用。约束瓶颈是PCIe带宽，而非预测质量。

只计算一个 token，因此 MoE GEMM 太小，无法掩盖侧流 PCIe 传输。投机复制因此会与需求复制争用同一条链路，而在严格的容量上限下，它们可能会驱逐下一步仍然需要的专家。实测流量让这一限制变得具体：在 Qwen3-30B-A3B 上，相邻解码步骤之间，每一层 top-8 专家中只有 44–48% 会沿用。即使有完美的上一步缓存，每个 token 仍必须流式传输约 1.8 GiB 的专家权重。在我们的 RTX 3090 上，主机到设备带宽约为 25GB/s，仅这一强制流量就将解码速度限制在约 13–16 tok/s。这条管道已经被占满；预测可以重新安排流量，但无法拓宽管道。

这一结果改变了我们使用路由信息的方式。在这种场景下，路由不是用于预取的延迟信号；它是一种内存信号，用于决定常驻专家集应有多大。一旦专家分页将部分模型从GPU常驻内存中释放出来，下一个问题就是如何花掉这些释放出来的内存：让更多专家常驻，还是把这些字节给KV缓存？这个分配问题正是MV-WSA所解决的（§3.3–§3.5），也是我们在§5.3中所评估的。

5 评估

评估遵循论文的论证。首先，WiSP不应改变模型的计算，因此我们检查输出保持性（第5.1节）。其次，如果专家驻留是一个工作集问题，那么在相同的GPU预算下，路由感知分页应比静态层级卸载移动更少的数据；我们在物理受限的24GiB RTX 3090上对此进行测试（第5.2节）。第三，由于专家内存和KV内存共享一个存储，我们评估MV-WSA是否选择了比固定或内存盲策略更好的划分，既作为启动配置器，也作为实时调整大小控制器（第5.3节）。

实验设置与范围。Qwen3-30B-A3B 和 Kimi-VL-A3B 的主要 iso-VRAM 吞吐量结果（图 3、表 2）是在一块物理受限的消费级 GPU 上测量的：单块 24 GiB 的 NVIDIA RTX 3090，运行在真实的 PCIe 4.0 x16 链路上（实测主机到设备带宽为≈25GB/s），vLLM 0.11.2，*eager* 模式，温度 0。这两个模型在 BF16 下均超过 24 GiB（Qwen 3 ≈57 GiB，Kimi-VL ≈31GiB），因此分页是由硬件强制触发的，而非模拟的。H100 的结果则有不同目的：MiniMax-M2 和 Jamba 是对超过单块 94 GiB H100 的模型进行能力检查，而 OLMoE MV-WSA 启动配置器测试则在受控的 8 GiB 预算下隔离分配策略。扩散实验使用独立的 PyTorch 分页器，因此我们在那里报告正确性和常驻内存行为，而非服务延迟。

5.1 分页保持输出不变

在温度为0时，WiSP在我们测试的每一个上限下都与未修改的引擎匹配，这是性质P1的预期结果：融合MoE内核保持不变，且每个被路由的专家都从字节相等的权重中读取。我们还在扩散模型上检查了该机制，那里的比较更为严格，因为我们可以检查整个生成轨迹：DiT-MoE的潜变量在整个上限扫描中匹配，最大绝对差为零，而LLaDA-MoE的补全在token上完全相同（第5.4节）。

5.2 Iso-VRAM Decode Throughput versus Static Offload

Table 2 summarizes the main system cases. Qwen3 and Kimi-VL are matched-budget throughput comparisons on the 24 GiB edge GPU. Jamba and MiniMax-M2 are capability checks on models whose native serving paths fail or become impractical under the stated memory constraints. The diffusion mechanism-only cases have no serving-throughput baseline and are reported in Appendix F.

Figure 3 is the main systems result on real edge hardware. On the 24 GiB RTX 3090, both WiSP and the static-offload baseline run under the same GPU budget, but they spend the bytes differently. Static offload streams the expert layer in bulk, so its decode throughput stays near 4–5 tok/s. WiSP keeps a routed expert working set resident, so throughput rises as the cap grows: 6.0, 7.6, and 9.9 tok/s for cap 8, 16, and 32. The result is a $\approx$1.8–2.0$\times$ speedup in the sub-fitting range this card can actually exercise. A 24 GiB card cannot reach the full-residency crossover; we only claim the regime where the model does not fit.

Kimi-VL gives a second check on the same mechanism in a different model family. It is a MoE vision–language model with a dense vision tower and a routed MoE language backbone, yet WiSP uses the same vLLM plug-in without model-specific code. At matched budgets, it improves decode throughput by 1.58–1.79$\times$ over static offload (Table 2) while preserving outputs on both text and image prompts (Section 5.4).

Scaling to MiniMax-M2: a capability check. We also run WiSP on the 229B-parameter FP8 MiniMax-M2 on one 94 GiB H100. This is a capability check, not a throughput claim. WiSP serves the model with coherent output while keeping only a 7–14 GiB expert scratch on the GPU. Stock vLLM can also serve the model on this card with enough static offload, and is faster, but lowering its GPU footprint requires more offloaded and pinned/staged host memory until host RAM becomes the bottleneck. WiSP's point here is different: the full expert set lives once in host memory, while the GPU holds only the resident scratch. A clean iso-GPU-VRAM comparison where this tradeoff pays off requires a physically smaller card, which we leave to future hardware work.

The throughput result is only the expert side of the trade. The working-set view says that the expert cache and the KV cache share one store: shrinking the resident expert cap leaves more GPU memory for on-device KV, while growing the expert cap reduces the KV pool available to the scheduler. Push the expert cap too high and admission can fail because the engine has no usable KV pool. Table 4 measures the reverse move online, reclaiming idle KV capacity and giving those bytes back to resident experts. This is the allocation problem MV-WSA is designed to solve.

5.3 Allocating the Budget: MV-WSA

MV-WSA chooses the split between resident experts and on-device KV. We evaluate it in two settings. First, as a static startup configurator, it chooses a fixed split before serving and we compare that split against fixed and KV-blind policies on real `vllm serve` (Table 3). Second, as a live controller, it physically moves bytes between the expert scratch and the KV pool while requests are running (Table 4).

Static configurator, real serving. We run one `vllm serve` instance per arm and replay `os` sessions from AgentInstruct (Zeng et al., 2024), the agent-trajectory dataset whose `os`/`db`/`webshop` splits are collected on AgentBench environments (Liu et al., 2024a), at concurrency 4. Each session is prepended with a distinct $\sim$1.2–1.5k-token pseudo-document, so concurrent requests hold separate KV state rather than sharing a cached prefix. All arms use the same total GPU budget and host CPU-swap; only the expert–KV split changes.

We compare four policies: **MV-WSA**, which uses the configurator split $\hat{f}$ subject to the admission floor; **50–50**, an even split of the pageable pool; **KV-blind prop.**[1], an expert-working-set-proportional split that ignores KV demand; and **naive offload**, vanilla `--cpu-offload-gb` with WiSP off. Table 3 runs this on OLMoE-1B-7B at an 8 GiB budget, a KV-pressured regime where wasting KV capacity shows up immediately in TTFT.

Table 3 shows the two failure modes. **KV-blind prop.** gives the most memory to experts and therefore has the fastest decode, but it leaves only 0.5 GiB for KV; TTFT rises to 19.8 s and turn wall reaches 22.0 s. **50–50** leaves more KV, but its smaller expert cap slows decode to 29.2 tok/s. **Naive offload** streams every expert and pays on both axes. **MV-WSA** balances the two pools: it matches the best TTFT, keeps decode well above 50–50, and has the lowest median turn wall.

The admission floor is what makes the right split feasible. The KV-blind prior in Table 3 shows the soft version of KV starvation: the engine runs, but TTFT collapses. The hard version is a startup failure. If an allocator pushes the split too far toward experts, the remaining KV pool may be too small to admit even one requested context, and the engine can refuse to initialize. The admission floor in Eq. (4) prevents this by clamping the split to the most expert-heavy allocation that still satisfies the scheduler's KV requirement.

[1]This is a deliberately KV-blind prior, not FluxMoE (Liu et al., 2026), which targets high-batch stream-and-evict overlap.

5.2 等 *VRAM* 条件下的解码吞吐量与静态卸载对比

表2总结了主要的系统案例。Qwen3和Kimi-VL是在24GiB边缘GPU上进行的匹配预算吞吐量比较。Jamba和MiniMax-M2是对那些原生服务路径在所述内存约束下失败或变得不切实际的模型进行的能力检查。仅涉及扩散机制的那些案例没有服务吞吐量基线，报告在附录F中。

图 3 是真实边缘硬件上的主要系统结果。在 24GiB RTX 3090 上，WiSP 和静态卸载基线在相同的 GPU 预算下运行，但它们对字节的使用方式不同。静态卸载会批量流式传输专家层，因此其解码吞吐量保持在 4–5 tok/s 附近。WiSP 则保持一个被路由的专家工作集常驻，因此吞吐量会随着上限增长而上升：当上限为 8、16 和 32 时，分别为 6.0、7.6 和 9.9 tok/s。其结果是，在这张卡实际能够发挥作用的 sub-fitting 范围内实现了 $\approx$1.8–2.0$\times$ 的加速。24GiB 显卡无法达到完全驻留的交叉点；我们只声称模型无法容纳时的这一区间。

Kimi-VL 在另一个模型家族中对同一机制给出了第二次验证。它是一个 MoE 视觉语言模型，具有密集视觉塔和路由式 MoE 语言主干，但 WiSP 使用相同的 vLLM 插件，无需模型特定代码。在匹配预算下，它将解码吞吐量相较静态卸载提高了 1.58–1.79$\times$ （表 2），同时在文本和图像提示上保持输出不变（5.4 节）。

扩展至MiniMax-M2：能力检验。我们还在单张94 GiB H100上对229B参数的FP8 MiniMax-M2运行WiSP。这是一项能力检验，而非吞吐量声明。WiSP在GPU上仅保留7–14 GiB的专家暂存空间，即可为该模型提供连贯输出。原生vLLM在足够静态卸载的情况下也能在此卡上服务该模型，且速度更快，但要降低其GPU占用则需要更多卸载和固定/暂存的主机内存，直至主机RAM成为瓶颈。WiSP在此处的意义不同：完整的专家集合仅驻留于主机内存一次，而GPU仅保留常驻暂存空间。要在这种权衡能带来收益的场景下进行干净的等GPU显存比较，需要一张物理上更小的卡，我们将此留待未来的硬件工作。

吞吐量结果只是权衡中专家侧的一面。工作集视角认为，专家缓存和 KV 缓存共享同一个存储：缩小常驻专家上限会为设备上的 KV 留出更多 GPU 内存，而增大专家上限则会减少调度器可用的 KV 池。将专家上限推得过高，准入可能会失败，因为引擎没有可用的 KV 池。表 4 在线测量了反向操作，即回收空闲的 KV 容量，并将这些字节返还给常驻专家。这就是分配问题 MV-WSA 旨在解决。

5.3 预算分配：*MV-WSA*

MV-WSA选择常驻专家与设备端KV之间的划分。我们在两种设置下对其进行评估。首先，作为静态启动配置器，它在服务前选择一个固定划分，我们在真实的 vllm serve上将该划分与固定策略和KV盲策略进行比较（表3）。其次，作为实时控制器，它在请求运行期间在专家暂存空间和KV池之间物理移动字节（表4）。

静态配置器，真实服务。我们为每个分支运行一个 vllm serve 实例，并以并发度 4 回放来自 AgentInstruct（Zeng et al., 2024）的 os 会话，该智能体轨迹数据集的 os/db/webshop 划分是在 AgentBench 环境（Liu et al., 2024a）上收集的。每个会话前都会附加一个不同的 $\sim$1.2–1.5k token 伪文档，因此并发请求持有独立的 KV 状态，而不是共享缓存前缀。所有分支使用相同的总 GPU 预算和主机 CPU 交换；仅专家–KV 划分发生变化。

我们比较四种策略：MV-WSA，它使用配置器划分 $\hat{f}$，并受准入下限约束；50–50，即可分页池的均等划分；KV-blind prop.[1]，，一种忽略 KV 需求的专家工作集比例划分；以及朴素卸载，即关闭 WiSP 的原版 --cpu-offload-gb。表 3 在 8GiB 预算下于 OLMoE-1B-7B 上运行该实验，这是一个 KV 受压制的场景，浪费 KV 容量会立即体现在 TTFT 中。

表 3 展示了两种失败模式。KV-blind prop. 将最多的内存分配给专家，因此解码速度最快，但仅为 KV 留下 0.5 GiB；TTFT 上升至 19.8 秒，轮次墙钟时间达到 22.0 秒。50–50 留下了更多 KV，但其较小的专家上限将解码速度拖慢至 29.2 tok/s。Naive offload 会流式传输每个专家，并在两个维度上都付出代价。MV-WSA 平衡了两个池：它达到了最佳 TTFT，将解码速度保持在远高于 50–50 的水平，并且具有最低的中位轮次墙钟时间。

准入下限正是使正确的拆分方案可行的关键。表3中的KV盲先验展示了KV饥饿的软性版本：引擎仍在运行，但TTFT崩溃。硬性版本则是启动失败。如果分配器将拆分过度推向专家侧，剩余的KV池可能太小，连一个请求的上下文都无法准入，引擎可能拒绝初始化。式(4)中的准入下限通过在满足调度器KV需求的前提下将拆分钳制到专家最重的分配方案，从而防止了这种情况。

[1]这是一个刻意采用 KV-blind 的先验方法，而非 FluxMoE（Liu et al., 2026），后者针对的是高批量流式与驱逐重叠。

Table 2. Main results across the full-system cases. Where a throughput comparison is possible we match GPU-memory budgets: *naive offload* is vLLM's static `--cpu-offload-gb` (offloaded weight bytes, in GiB) and *WiSP paging* keeps a cap-sized resident expert scratch (cap, in experts/layer); both columns report decode *tok/s* at that budget. EDGE = measured on a physically constrained 24 GiB RTX 3090. REAL/CAP. = the model exceeds one 94 GiB H100 or hits an unsupported native path, so the native baseline does not run (×); these rows are capability checks rather than throughput claims and have no speedup ("—").

		naive offload		WiSP paging		
Setting	GPU budget	GiB	tok/s	cap	tok/s	Speedup
MoE LLM — Qwen3-30B-A3B (BF16), 128 experts, top-8, on a 24 GiB RTX 3090 / PCIe 4.0 (cf. Figure 3)						
EDGE	≈20 GiB	50	4.17	16	7.62	1.83×
EDGE	≈22 GiB	44	4.93	32	**9.93**	**2.01**×
MoE VLM — Kimi-VL-A3B (BF16), 64 experts, top-8, on a 24 GiB RTX 3090 / PCIe 4.0						
EDGE	≈14.7 GiB	20	5.00	16	8.97	1.79×
EDGE	≈8.8 GiB	24	4.23	8	6.69	1.58×
Hybrid SSM–MoE — Jamba-v0.1 (52B, 103 GiB BF16), 16 experts, top-2						
REAL	94 GiB H100	full residency: × OOM (103>94)		4	5.73	—
REAL	94 GiB H100	`cpu-offload-gb` (any): × assertion		8	9.80	—
MoE LLM — MiniMax-M2 (229B FP8), 256 experts, top-8						
CAP.	94 GiB H100	offload ≥180: × host OOM (256 GiB cgroup)		32	3.70 (14 GiB exp.)	—

Table 3. MV-WSA startup configurator (*static* split, fixed for the run) on real `vllm serve`, iso-VRAM, AgentInstruct (Zeng et al., 2024) `os`, concurrency 4, OLMoE-1B-7B at an 8 GiB budget. Every arm gets the same total GPU budget and host CPU-swap; only the expert–KV split changes. *Turn wall* is median wall-clock time per turn (prefill+decode), lower is better.

Arm (split)	cap	KV (GiB)	TTFT med	decode	turn wall (s)
MV-WSA	21	2.6	**2.9**	40.6	**6.3**
50–50	18	3.2	2.9	29.2	7.0
KV-blind prop.	35	0.5	19.8	**56.2**	22.0
naive offload	—	—	21.3	30.7	25.5

Table 4. Live MV-WSA dual-resize (*dynamic*: both pools resized *while* serving) vs. the fixed offline split, on a real 24 GiB RTX 3090, Qwen3-30B-A3B, AgentInstruct (Zeng et al., 2024) `os` and `db`, concurrency 4, 36 timed turns. Both arms replay the byte-identical trace; the only difference is the policy. Lower is better except decode. The dynamic arm reclaims idle KV into experts in one move (cap 32 → 35, KV 2.79 → 1.53 GiB) and reduces end-to-end time on both traces.

Trace	Arm	cap	KV pool	TTFT med (s)	decode (tok/s)	e2e (s)
os	fixed	32	2.79 GiB	89.31	3.17	1237.1
	dynamic	32→**35**	2.79→**1.53** GiB	**80.74**	3.14	**1155.0**
	dyn/fix	—	—	1.11×	0.99×	**1.07**×
db	fixed	32	2.79 GiB	95.46	3.02	1355.4
	dynamic	32→**35**	2.79→**1.53** GiB	**75.05**	**3.36**	**1138.4**
	dyn/fix	—	—	1.27×	1.11×	**1.19**×

Live dual-resize: reclaiming idle KV. The closed-form controller of §3.5 runs in the in-process engine and resizes both pools at drained barriers. On the 24 GiB RTX 3090, we replay two Qwen3 AgentInstruct traces (`os`, `db`) at concurrency 4. In both traces, the fixed offline split reserves more KV than the workload actually uses: the KV pool peaks at only ≈44% of its allocation each round. The controller observes this after the first drained round and moves the unused KV budget to resident experts (cap 32 → 35, KV 2.79 → 1.53 GiB), then holds that split with zero preemptions.

Table 4 shows the effect. Dynamic resizing reduces end-to-end time by 1.07–1.19× over the fixed offline split at the same GPU budget, and improves TTFT by 1.11–1.27×. Decode improves on `db` and is essentially unchanged on `os`. A separate resize-cycle check confirms property (D2): outputs remain byte-identical across a shrink-KV/grow-experts and grow-KV/shrink-experts cycle. The gain is regime-dependent: when the workload is already KV-pressured, as in Table 3, the static configurator is already near the right split and there is little idle KV to reclaim.

5.4 Generality across Architectures and Modalities

The expert-paging mechanism needs only routed experts, so it carries beyond text-only MoE LLMs. On **Kimi-VL-A3B**, the same vLLM plug-in pages the MoE language backbone with no model-specific code, preserves outputs on both text and image prompts, and improves decode throughput at matched budgets (Section 5.2). On the hybrid **Jamba-v0.1** (52B; Mamba+attention+MoE), WiSP serves the model on one 94 GiB GPU in a regime where stock vLLM cannot: full residency exceeds VRAM, and the CPU-offload path is unsupported for Jamba's recurrent state. This is a capability claim scoped to the hybrid model. For MoE diffusion models without an autoregressive KV cache, the full expert–KV allocation problem does not apply, but the pager still does. A roughly 150-line PyTorch wrapper pages **DiT-MoE** and

表 2. 全系统场景下的主要结果。在可以进行吞吐量比较的情况下，我们匹配 GPU 内存预算：朴素卸载是 vLLM 的静态 --cpu-offload-gb（卸载的权重字节数，单位为 GiB），而 WiSP 分页保持一个上限大小的常驻专家暂存区（上限，单位为专家数/层）；两列均报告该预算下的解码 tok/s。EDGE = 在物理受限的 24 GiB RTX 3090 上测量。REAL/CAP。 = 模型超过单个 94 GiB H100 或触及不受支持的原生路径，因此原生基线无法运行（×）；这些行是能力检查，而非吞吐量声明，且没有加速比（"—"）。

		朴素卸载		WiSP 分页		
设置	GPU 预算	GiB	tok/s	cap	tok/s	加速比
MoE LLM — Qwen3-30B-A3B（BF16），128个专家，top-8，在24 GiB RTX 3090 / PCIe 4.0上（参见图3）						
EDGE	≈20 GiB	50	4.17	16	7.62	1.83×
EDGE	≈22 GiB	44	4.93	32	**9.93**	**2.01**×
MoE VLM — Kimi-VL-A3B（BF16），64 个专家，top-8，运行于 24 GiB RTX 3090 / PCIe 4.0						
EDGE	≈14.7 GiB	20	5.00	16	8.97	1.79×
EDGE	≈8.8 GiB	24	4.23	8	6.69	1.58×
混合 SSM–MoE — Jamba-v0.1（52B，103 GiB BF16），16 个专家，top-2						
REAL	94 GiB H100	完全驻留： ×OOM（103>94）		4	5.73	—
REAL	94 GiB H100	cpu-offload-gb（任意）： ×断言		8	9.80	—
MoE LLM — MiniMax-M2（229B FP8），256 个专家，top-8						
CAP.	94 GiB H100	offload ≥180： × 主机 OOM（256 GiB cgroup）		32	3.70（14 GiB 实验值）	—

表 3. 在真实 vllm serve 上的 MV-WSA 启动配置器（静态划分，运行期间固定），等 VRAM，AgentInstruct（Zeng 等，2024）os，并发 4，OLMoE-1B-7B 在 8GiB 预算下。每个分支获得相同的总 GPU 预算和主机 CPU-swap；只有专家–KV 划分发生变化。轮次墙为每轮的中位墙钟时间（prefill+decode），越低越好。

分支（划分）	cap	KV (GiB)	TTFT 中位数	解码	轮次墙 (s)
MV-WSA	21	2.6	**2.9**	40.6	**6.3**
50–50	18	3.2	2.9	29.2	7.0
KV盲比例	35	0.5	19.8	**56.2**	22.0
朴素卸载	—	—	21.3	30.7	25.5

表4. 实时MV-WSA双重调整大小（动态：服务期间两个池均调整大小）与固定离线划分的对比，在真实的24 GiB RTX 3090、Qwen3-30B-A3B、AgentInstruct (Zeng等，2024) os和db上，并发4，36个计时轮次。两组均重放字节完全相同的轨迹；唯一的区别是策略。除解码外，越低越好。动态组一步将空闲KV回收给专家（上限 32 → 35，KV 2.79 → 1.53 GiB），并减少了两条轨迹上的端到端时间。

追踪	Arm	cap	KV池	TTFT中位数（秒）	解码（tok/s）	端到端（秒）
os	固定	32	2.79 GiB	89.31	3.17	1237.1
	动态	32→**35**	2.79→**1.53** GiB	**80.74**	3.14	**1155.0**
	动态/固定	—	—	1.11×	0.99×	**1.07**×
db	固定	32	2.79 GiB	95.46	3.02	1355.4
	动态	32→**35**	2.79→**1.53** GiB	**75.05**	**3.36**	**1138.4**
	动态/固定	—	—	1.27×	1.11×	**1.19**×

实时双重重调整：回收空闲KV。§3.5中的闭式控制器在进程内引擎中运行，并在排空屏障处调整两个池的大小。在24 GiB的RTX 3090上，我们以并发度4回放两条Qwen3 AgentInstruct轨迹（os、db）。在两条轨迹中，固定的离线拆分预留的KV都多于工作负载实际使用的量：KV池每轮峰值仅达到其分配量的 ≈44%。控制器在第一个排空轮次后观察到这一点，将未使用的KV预算转移到常驻专家（上限 32 → 35，KV 2.79 → 1.53GiB），随后保持该拆分且零抢占。

表4显示了效果。在相同的GPU预算下，动态调整大小相比固定的离线划分，将端到端时间减少了1.07–1.19× 倍，并将TTFT提高了1.11–1.27×。解码在db上有改善，在os上基本不变。单独的调整周期检查确认了性质(D2)：在收缩KV/增长专家和增长KV/收缩专家的周期中，输出保持字节完全相同。增益取决于工况：当工作负载已经处于KV压力下时，如表3所示，静态配置器已经接近正确拆分，且几乎没有空闲的 KV 可供回收。

5.4 跨架构与模态的通用性

专家分页机制只需要被路由的专家，因此其适用范围超越了纯文本的MoE大语言模型。在Kimi-VL-A3B上，同一个vLLM插件即可对MoE语言主干进行分页，无需任何模型特定代码，在文本和图像提示上均保持输出不变，并在匹配预算下提升了解码吞吐量（第5.2节）。在混合架构Jamba-v0.1（52B；Mamba+attention+MoE）上，WiSP能够在单张94GiB GPU上服务该模型，而这是原生vLLM无法做到的：全量驻留会超出显存，且CPU卸载路径不支持Jamba的循环状态。这一能力声明仅限于该混合模型。对于没有自回归KV缓存的MoE扩散模型，完整的专家–KV分配问题并不适用，但分页器仍然有效。一个约150行的PyTorch封装即可对DiT-MoE和

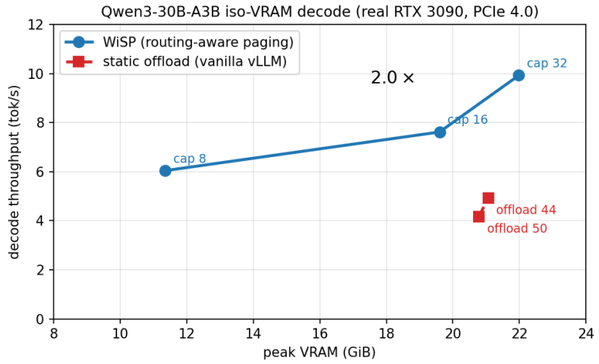


Figure 3. Iso-VRAM decode throughput on Qwen3-30B-A3B, measured on a real 24 GiB RTX 3090 (PCIe 4.0). When the model does not fit, routing-aware paging (WiSP) moves fewer bytes per step than static layer-level offload. At a matched ≈22 GiB budget, WiSP decodes ≈2.0× faster (cap 32: 9.93 vs. offload 44: 4.93 tok/s). Within the range this card can hold, increasing the expert cap improves WiSP throughput (cap 8/16/32 → 6.0/7.6/9.9 tok/s), while the routing-blind baseline remains bandwidth-limited. A 24 GiB card cannot reach the full-footprint budget where the model fits and static residency wins; WiSP targets the sub-fitting regime, not a universal speedup.

LLaDA-MoE-7B; generation remains bit-/token-identical to full residency, while resident expert memory drops by up to $4\times$. These models also expose different working-set structures: DiT has a fully activated per-forward stream with little cross-step reuse, while LLaDA has strong temporal locality across denoising steps. Appendix F reports the full mechanism-generalization results (Figures 4, 5).

6 Conclusion

Serving an MoE on a GPU that cannot keep it resident is a working-set problem: routed expert weights and the KV cache are two memory-demand streams competing for one small store. WiSP realizes this view for the low-concurrency regime. It pages routed experts through a drop-in, byte-identical vLLM plug-in, improves decode throughput by up to $2.0\times$ over static offload at matched VRAM on a real 24 GiB GPU, and carries the same expert-paging mechanism to a MoE VLM, a 52B hybrid model whose stock vLLM path cannot serve it on one GPU, and two diffusion MoEs as mechanism-generalization checks.

The main lesson is that in single-stream, memory-bound decode, routing prediction is not the speed lever. There is too little compute to hide PCIe transfers, so the routing signal is more useful as a memory signal: it tells us which expert working set is worth keeping resident. That makes allocation the central problem. MV-WSA splits one VRAM budget between resident experts and KV under an admission floor, and the live controller shows that this split can be changed while serving without changing model outputs. Porting the full estimator into the live serve loop, broadening the dynamic controller beyond the current proof of concept, and validating the MiniMax regime on physically smaller GPUs are the next steps.

References

Anthony, Q., Tokpanov, Y., Glorioso, P., and Millidge, B. BlackMamba: Mixture of experts for state-space models. *arXiv preprint arXiv:2402.01771*, 2024.

Cao, S., Liu, S., Griggs, T., Schafhalter, P., Liu, X., Sheng, Y., Gonzalez, J. E., Zaharia, M., and Stoica, I. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Denning, P. J. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

Denning, P. J. Virtual memory. *ACM Computing Surveys*, 2 (3):153–189, 1970.

Du, Z. et al. SiDA: Sparsity-inspired data-aware serving for efficient and scalable large Mixture-of-Experts models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Eliseev, A. and Mazur, D. Fast inference of Mixture-of-Experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.

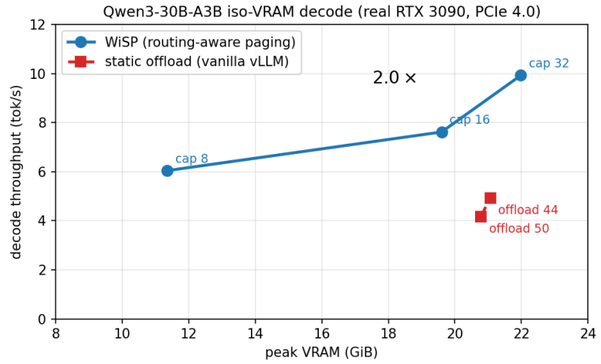


图 3. Qwen3-30B-A3B 在真实 24 GiB RTX 3090（PCIe 4.0）上的等 VRAM 解码吞吐量。当模型无法完全放入显存时，路由感知分页（WiSP）每步移动的字节数少于静态层级卸载。在匹配的 ≈22 GiB 预算下，WiSP 解码速度更快（cap 32：9.93 对比 offload 44：4.93 tok/s）。在该卡可容纳的范围内，提高专家上限可提升 WiSP 吞吐量（cap 8/16/32 → 6.0/7.6/9.9 tok/s），而路由盲基线仍受带宽限制。24 GiB 显卡无法达到模型完全放入且静态常驻胜出的全量预算；WiSP 针对的是次适配场景，而非通用加速。

LLaDA-MoE-7B进行分页；生成结果与全量驻留保持逐位/逐token完全一致，而驻留专家内存最多降低 $4\times$。这些模型还展现出不同的工作集结构：DiT具有完全激活的逐前向流，跨步复用很少，而LLaDA在去噪步骤间具有强时间局部性。附录F报告了完整的机制泛化结果（图4、图5）。

6 结论

在无法将其常驻的GPU上服务MoE是一个工作集问题：路由专家权重和KV缓存是两条竞争同一小块存储的内存需求流。WiSP在低并发场景下实现了这一视角。它通过一个即插即用、字节级一致的vLLM插件对路由专家进行分页，在真实的24GiB GPU上以匹配的显存将解码吞吐量相比静态卸载提升最高达 $2.0\times$，并将相同的专家分页机制推广到一个MoE VLM、一个52B混合模型（其原生vLLM路径无法在单GPU上服务）以及两个扩散MoE，作为机制泛化性验证。

主要教训在于，在单流、内存受限的解码中，路由预测并非提速的关键。可用于隐藏PCIe传输的计算量太少，因此路由信号更适合作为内存信号：它告诉我们哪些专家工作集值得常驻。这使得分配成为核心问题。MV-WSA在准入下限约束下，将一个VRAM预算分配给常驻专家和KV，而实时控制器表明，这一分配可以在服务过程中改变，而不改变模型输出。将完整的估计器移植到实时服务循环中、将动态控制器扩展到当前概念验证之外，并在物理尺寸更小的GPU上验证MiniMax方案，是接下来的步骤。

参考文献

Anthony, Q.、Tokpanov, Y.、Glorioso, P. 和 Millidge, B. BlackMamba：面向状态空间模型的混合专家。arXiv预印本 arXiv:2402.01771，2024。

Cao, S., Liu, S., Griggs, T., Schafhalter, P., Liu, X., Sheng, Y., Gonzalez, J. E., Zaharia, M., and Stoica, I. MoE-Lightning: 在内存受限GPU上的高吞吐量MoE推理。载于ACM编程语言与操作系统架构支持国际会议（ASPLOS），2025年。

DeepSeek-AI。DeepSeek-V3技术报告。arXiv预印本 arXiv:2412.19437，2024。Denning, P. J. 程序行为的工作集模型。ACM通讯，11(5):323–333，1968。Denning, P. J. 虚拟内存。ACM计算综述，2(3):153–189，1970。Du, Z.等。SiDA：面向稀疏性的数据感知服务，用于高效且可扩展的大型混合专家模型。载于机器学习系统会议论文集（MLSys），2024。Eliseev, A.和Mazur, D. 基于卸载的混合专家语言模型快速推理。arXiv预印本arXiv:2312.17238，2023。

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, 23:1–39, 2022.

Fei, Z., Fan, M., Yu, C., Li, D., and Huang, J. Scaling diffusion transformers to 16 billion parameters (DiT-MoE). *arXiv preprint arXiv:2407.11633*, 2024.

Gerganov, G. et al. llama.cpp. https://github.com/ggml-org/llama.cpp, 2023.

Huang, Y., Fang, Z., Luo, W., Wu, R., Chen, W., and Zheng, Z. DyMoE: Dynamic expert orchestration with mixed-precision quantization for efficient MoE inference on edge. *arXiv preprint arXiv:2603.19172*, 2026.

Hwang, R., Wei, J., Cao, S., Hwang, C., Tang, X., Cao, T., and Yang, M. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

Jiang, A. Q. et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Kimi Team. Kimi-VL technical report. *arXiv preprint arXiv:2504.07491*, 2025.

KVCache.AI. KTransformers: A flexible framework for cutting-edge LLM inference optimizations. https://github.com/kvcache-ai/ktransformers, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Li, D. et al. Aria: An open multimodal native mixture-of-experts model. *arXiv preprint arXiv:2410.05993*, 2024.

Lieber, O. et al. Jamba: A hybrid transformer-Mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

Liu, Q., He, C. Y., Jiang, H., Wang, Z., Zhao, A., and Lee, P. P. C. FluxMoE: Decoupling expert residency for high-performance MoE serving. *arXiv preprint arXiv:2604.02715*, 2026.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., and Tang, J. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024a. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024b.

Nie, S. et al. Large language diffusion models (LLaDA). *arXiv preprint arXiv:2502.09992*, 2025.

Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated Mixture-of-Experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Wu, Z. et al. DeepSeek-VL2: Mixture-of-experts vision-language models for advanced multimodal understanding. *arXiv preprint arXiv:2412.10302*, 2024.

Xue, L., Fu, Y., Lu, Z., Sun, C., Mai, L., and Marina, M. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.

Yang, S., Fan, X., Pan, M., Xi, H., Wang, Z., Sun, S., Keutzer, K., Han, S., Zaharia, M., Xu, C., and Stoica, I. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.

Yu, H., Cui, X., Zhang, H., Wang, H., and Wang, H. Taming latency-memory trade-off in MoE-based LLM serving via fine-grained expert offloading. In *European Conference on Computer Systems (EuroSys)*, 2026.

Zeng, A., Liu, M., Lu, R., Wang, B., Liu, X., Dong, Y., and Tang, J. AgentTuning: Enabling generalized agent abilities for LLMs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3053–3077, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.181. URL https://aclanthology.org/2024.findings-acl.181/.

Zheng, L. et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Zhong, S., Sun, Y., Liang, L., Wang, R., Huang, R., and Li, M. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *ACM/IEEE Design Automation Conference (DAC)*, 2025.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research (JMLR), 23:1–39, 2022.

Fei, Z., Fan, M., Yu, C., Li, D., and Huang, J. Scaling diffusion transformers to 16 billion parameters (DiT-MoE). arXiv preprint arXiv:2407.11633, 2024.

Gerganov, G. et al. llama.cpp. https://github.com/ggml-org/llama.cpp, 2023.

Huang, Y., Fang, Z., Luo, W., Wu, R., Chen, W., and Zheng, Z. DyMoE: Dynamic expert orchestration with mixed-precision quantization for efficient MoE inference on edge. arXiv preprint arXiv:2603.19172, 2026.

Hwang, R., Wei, J., Cao, S., Hwang, C., Tang, X., Cao, T., and Yang, M. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In ACM/IEEE International Symposium on Computer Architecture (ISCA), 2024.

Jiang, A. Q. et al. Mixtral of experts. arXiv preprint arXiv:2401.04088, 2024.

Kimi Team. Kimi-VL technical report. arXiv preprint arXiv:2504.07491, 2025.

KVCache.AI. KTransformers: A flexible framework for cutting-edge LLM inference optimizations. https://github.com/kvcache-ai/ktransformers, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In ACM Symposium on Operating Systems Principles (SOSP), 2023.

Li, D. et al. Aria: An open multimodal native mixture-of-experts model. arXiv preprint arXiv:2410.05993, 2024.

Lieber, O. et al. Jamba: A hybrid transformer-Mamba language model. arXiv preprint arXiv:2403.19887, 2024.

Liu, Q., He, C. Y., Jiang, H., Wang, Z., Zhao, A., and Lee, P. P. C. FluxMoE: Decoupling expert residency for high-performance MoE serving. arXiv preprint arXiv:2604.02715, 2026.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., and Tang, J. AgentBench: Evaluating LLMs as agents. In International Conference on Learning Representations (ICLR), 2024a. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: 一种面向KV缓存的无调优非对称2比特量化。载于《国际机器学习会议》(ICML)，2024b。

Nie, S. 等. 大型语言扩散模型 (LLaDA). arXiv 预印本 arXiv:2502.09992, 2025.

Peebles, W. and Xie, S. 基于 Transformer 的可扩展扩散模型. In IEEE/CVF International Conference on Computer Vision (ICCV), 2023. Qwen Team. Qwen3 技术报告. arXiv preprint arXiv:2505.09388, 2025.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. 极其庞大的神经网络：稀疏门控的专家混合层。载于《国际学习表征会议》(ICLR)，2017年。

Wu, Z. 等。DeepSeek-VL2：用于高级多模态理解的专家混合视觉语言模型。arXiv 预印本 arXiv:2412.10302, 2024。

Xue, L.、Fu, Y.、Lu, Z.、Sun, C.、Mai, L. 和 Marina, M. MoE-Infinity：在个人机器上具有稀疏感知专家缓存的高效 MoE 推理。arXiv 预印本 arXiv:2401.14361，2024。

Yang, S.、Fan, X.、Pan, M.、Xi, H.、Wang, Z.、Sun, S.、Keutzer, K.、Han, S.、Zaharia, M.、Xu, C. 和 Stoica, I. FreeToken：具有带宽自适应执行的高效边缘原生 MoE 服务。arXiv 预印本 arXiv:2608.16157，2026。

Yu, H.、Cui, X.、Zhang, H.、Wang, H. 和 Wang, H. 通过细粒度专家卸载来驾驭基于 MoE 的 LLM 服务中的延迟-内存权衡。在欧洲计算机系统会议（EuroSys），2026。Zeng, A.、Liu, M.、Lu, R.、Wang, B.、Liu, X.、Dong, Y. 和 Tang, J. AgentTuning：为 LLM 启用广义智能体能力。在计算语言学协会发现：ACL 2024，第 3053–3077 页，泰国曼谷，2024。计算语言学协会。doi: 10.18653/v1/2024.findings-acl.181。URL https://aclanthology.org/2024.findings-acl.181/。

Zheng, L. 等. SGLang：结构化语言模型程序的高效执行. 载于《神经信息处理系统进展》(NeurIPS)，2024.

Zhong, S., Sun, Y., Liang, L., Wang, R., Huang, R., and Li, M. HybriMoE：用于高效MoE推理的CPU-GPU混合调度与缓存管理。载于ACM/IEEE设计自动化会议（DAC），2025年。

Zhu, F., You, Z., Xing, Y., Huang, Z., et al. LLaDA-MoE: A sparse MoE diffusion language model. *arXiv preprint arXiv:2509.24389*, 2025.

A Related Work

WiSP's conceptual basis is Denning's working-set model and thrashing theory (Denning, 1968; 1970); our contribution is to recognize low-resource MoE serving as a direct instance and use that vocabulary to reason about expert residency, KV capacity, and admission together. On the systems side, PagedAttention in vLLM (Kwon et al., 2023) and RadixAttention in SGLang (Zheng et al., 2024) brought a paging and prefix-sharing discipline to the KV reference stream; we bring the same discipline to the expert stream and treat the two as co-contending working sets over one store.

A line of batch-oriented systems serves MoEs on memory-constrained GPUs by overlapping transfer with compute: MoE-Lightning (Cao et al., 2025) pipelines paged weights and reports up to $10.3\times$ on a T4, SiDA-MoE (Du et al., 2024) uses an offline hash predictor of active experts (approximate, $\sim 1\%$ accuracy drop), and Mixtral-offloading (Eliseev & Mazur, 2023) streams experts for a single stream. These target high throughput at large batch, where there is ample compute to hide transfers. A parallel line predicts or caches experts to cut miss latency rather than stream blindly: MoE-Infinity (Xue et al., 2024) keeps a sparsity-aware expert cache driven by activation tracking on personal machines; Pre-gated MoE (Hwang et al., 2024) co-designs the model to pre-compute the next layer's experts one step ahead; HybriMoE (Zhong et al., 2025) splits expert work across CPU and GPU with an impact-driven prefetch; FineMoE (Yu et al., 2026) retrieves per-request expert maps by semantic and trajectory similarity to guide prefetching; and DyMoE (Huang et al., 2026) assigns experts mixed precision (down to skipping) to shrink edge transfers. Most share WiSP's premise that expert residency is the bottleneck, but spend the routing signal on prefetching or overlap for speed—which Section 4 shows is a poor fit for single-stream decode, where no compute hides a transfer. WiSP instead spends the signal on sizing and treats experts and KV as one budget. The closest concurrent work is FluxMoE (Liu et al., 2026), which, like WiSP, builds on vLLM and reclaims expert VRAM for the KV cache, but adopts a stream-and-evict policy—paging an expert in just before its layer and evicting it immediately, pipelined with layer compute—to maximize high-batch throughput. WiSP differs in four ways: it targets the low-concurrency, cache-and-reuse operating point, where Section 4 shows there is no compute to overlap; it ships as a pure plug-in over the unmodified engine with no virtual-memory machinery; rather than a fixed, overlap-tuned residency plan it *allocates* the expert↔KV split by marginal value (MV-WSA), online and subject to an admission floor, and physically resizes both pools while serving; and it contributes the working-set framing, the measurement of when prediction does and does not pay off, and the cross-architecture breadth. The systems are

Zhu, F., You, Z., Xing, Y., Huang, Z., et al. LLaDA-MoE：一种稀疏MoE扩散语言模型。arXiv预印本arXiv:2509.24389，2025年。

相关工作

WiSP的概念基础是Denning的工作集模型与抖动理论（Denning，1968；1970）；我们的贡献在于将低资源MoE服务识别为该模型的一个直接实例，并运用这套术语来统一推理专家驻留、KV容量与准入问题。在系统层面，vLLM中的PagedAttention（Kwon et al.，2023）和SGLang中的RadixAttention（Zheng et al.，2024）为KV引用流引入了分页与前缀共享的纪律；我们将同样的纪律引入专家流，并将二者视为在同一存储上共同竞争的工作集。

一系列面向批处理的系统通过在内存受限的GPU上重叠传输与计算来服务MoE：MoE-Lightning（Cao等，2025）对分页权重进行流水线处理，并报告在T4上可达 $10.3\times$，SiDA-MoE（Du等，2024）使用活跃专家的离线哈希预测器（近似，精度下降 $\sim$1%），而Mixtral-offloading（Eliseev & Mazur，2023）为单一流流式传输专家。这些系统面向大批量下的高吞吐量，此时有充足的计算来隐藏传输。另一条平行研究路线通过预测或缓存专家来减少未命中延迟，而非盲目流式传输：MoE-Infinity（Xue等，2024）在个人机器上通过激活追踪维护稀疏感知的专家缓存；Pre-gated MoE（Hwang等，2024）协同设计模型以提前一步预计算下一层的专家；HybriMoE（Zhong等，2025）通过影响驱动的预取将专家工作分配到CPU和GPU上；FineMoE（Yu等，2026）通过语义和轨迹相似性检索每请求的专家映射以指导预取；DyMoE（Huang等，2026）为专家分配混合精度（直至跳过）以缩小边缘传输。大多数与WiSP共享专家驻留是瓶颈这一前提，但将路由信号用于预取或重叠以提升速度——第4节表明,这对于单流解码并不适合，因为没有计算可以隐藏传输。WiSP反而将信号用于容量规划，并将专家和KV视为统一预算。最接近的同期工作是FluxMoE（Liu等，2026），它像WiSP一样构建于vLLM之上，并回收专家VRAM用于KV缓存，但采用流式驱逐策略——在其层之前调入专家并立即驱逐，与层计算流水线化——以最大化大批量吞吐量。WiSP有四点不同：它面向低并发、缓存复用的工作点，第4节表明此时没有计算可重叠；它以纯插件形式发布在未修改的引擎之上，无需虚拟内存机制；它不采用固定的、针对重叠调优的驻留方案，而是通过边际价值（MV-WSA）在线分配专家↔KV的划分，并受准入下限约束，同时在服务过程中物理调整两个池的大小；它还贡献了工作集框架、对预测何时有效何时无效的测量，以及跨架构的广度。这些系统是

complementary across the concurrency axis.

Concurrent with this work, FreeToken (Yang et al., 2026) co-designs a full edge-native MoE serving stack—pipelined full-layer expert loading for prefill, semantic-aware state caching for agentic context edits, and an elastic expert/KV memory mechanism—and, at decode, serves residual expert misses through a second channel: executing missing experts in place on the CPU, splitting each layer's misses between PCIe fill and CPU compute in proportion to profiled bandwidths. This adds a degree of freedom that allocation alone cannot supply under the single-stream PCIe ceiling of Section 4: where WiSP can only fetch a missing expert, FreeToken may also compute it where it resides. The two designs are complementary along the allocation axis: FreeToken provides a runtime mechanism for reconfiguring the expert/KV split but does not specify a policy for choosing or adapting that split, which is the question MV-WSA answers, and extending MV-WSA's marginal-value rule to price a second service channel (fetch versus compute) is a natural next step.

Prior work on MoE routing prediction and speculative loading motivates the predictor we build in Section 4; we evaluate it directly and report a negative speed result for the low-concurrency regime, which clarifies that the signal is better spent on memory than latency. KV-cache compression methods such as KIVI (Liu et al., 2024b) reduce the other side of the joint working set and compose with WiSP rather than competing with it. Finally, WiSP is evaluated across MoE LLMs (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024; DeepSeek-AI, 2024; Qwen Team, 2025), MoE VLMs (Wu et al., 2024; Kimi Team, 2025; Li et al., 2024), a hybrid SSM-MoE (Lieber et al., 2024; Anthony et al., 2024), and MoE diffusion models for images (Peebles & Xie, 2023; Fei et al., 2024) and language (Nie et al., 2025; Zhu et al., 2025); we use these only as test subjects and change none of them.

B Dynamic Dual-Resize Control Law

Recall $a = LM_e$. At a drained barrier the conserved budget is measured directly, $B = \kappa b + Ca$, and the target sizes $\tilde{C}, \tilde{\kappa}$ are

$$\tilde{\kappa} = \max\big(\kappa_{\min}, \lceil (1+h)\, p \rceil\big), \tag{6}$$

$$\tilde{C} = \mathrm{clip}\big(\lfloor (B - \tilde{\kappa}\, b)/a \rfloor,\, C_{\min},\, C_{\max}\big), \tag{7}$$

$$\tilde{\kappa} \leftarrow \lfloor (B - \tilde{C}\, a)/b \rfloor, \tag{8}$$

where p is the burst's peak KV occupancy (blocks actually used) and h the KV headroom. Equation (6) promotes the admission floor (4) from a one-shot startup constraint to a continuously enforced invariant; (7) is the equimarginal "remainder to experts" rule for a KV step curve; and the recompute (8) reclaims any slack the cap clamp leaves, so the move never overcommits ($\tilde{C}\, a + \tilde{\kappa}\, b \le B$).

The *expert-scratch resize* grows or shrinks the per-layer GPU scratch S to $\tilde{C}$ slots, re-paging now-resident experts from W^{cpu} and dropping victims on a shrink, with π, π^{-1} and the LRU ticks resized alongside. The *KV-pool resize* reallocates the worker's per-layer KV tensors to $\tilde{\kappa}$ blocks and rebuilds the scheduler's shared block pool in place, preserving the cached `null_block` identity, done only when the pool is fully drained. To keep the transient footprint at $\max(\text{old}, \text{new}) \le B$ rather than their sum, the controller always frees the shrinking side before growing the other (on an expert grow: shrink KV, then grow scratch; on a shrink: the reverse). Every move is wrapped so a failed resize is skipped—the engine is left in its prior, usable state—and a cap dead-zone Δ suppresses moves smaller than Δ experts so the split is stable under round-to-round noise.

C Implementation Details

WiSP registers through the standard `vllm.general_plugins` entry point, so a single `pip install` suffices; the plug-in fires in every vLLM process before any model class is imported and patches three methods on the fused-MoE module—weight creation, post-load processing, and the forward. Weight creation allocates the expert masters on pinned host memory (this is what avoids the load-time GPU spike); post-load processing builds the per-layer C-slot scratch and the maps π, π^{-1}; the forward runs Algorithm 1 and dispatches the unmodified kernel with π as `expert_map`. The FP8 block-quantized path pages four tensors per expert (the weights and their block scales) and forces the Triton backend to bypass DeepGEMM's load-time post-processing, which assumes full residency. The dynamic controller's two primitives are `resize_layer_caps` (grow or shrink the expert scratch, re-paging from the host master) and a KV-pool reallocation that rebuilds the scheduler's block pool in place (Appendix B).

D Serving Design Space

This appendix expands the placement of WiSP among existing systems summarized in Section 2.3. Table 5 compares WiSP against prior autoregressive MoE-LLM serving systems along the axes that matter for our setting (routing-awareness, byte-identity, delivery, and target regime), and Table 6 lists the native serving stacks of the non-AR MoE modalities and how WiSP attaches to each.

E The Co-Activation Predictor

Section 4 uses a lightweight routing predictor to test whether expert prediction can hide page faults in low-concurrency

在并发轴上互补的。

与本工作同期，FreeToken（Yang 等，2026）共同设计了一套完整的边缘原生 MoE 服务栈——用于预填充的流水线式全层专家加载、面向智能体上下文编辑的语义感知状态缓存，以及弹性专家/KV 内存机制——并且在解码阶段，通过第二条通道来处理剩余的专家未命中：在 CPU 上就地执行缺失的专家，按照经性能剖析得到的带宽比例，将每一层的未命中在 PCIe 填充与 CPU 计算之间进行拆分。这增加了一个仅靠分配无法提供的自由度，而这是在 Section 4 的单流 PCIe 上限下无法实现的：WiSP 只能获取缺失的专家，而 FreeToken 还可以在专家所在处直接计算它。这两种设计在分配轴上互补：FreeToken 提供了一种用于重新配置专家/KV 划分的运行时机制，但并未给出选择或自适应调整该划分的策略，而这正是 MV-WSA 所回答的问题；将 MV-WSA 的边际价值规则扩展到为第二条服务通道（获取与计算）定价，则是自然的下一步。

先前关于MoE路由预测和推测性加载的工作启发了我们在第4节中构建的预测器；我们对其进行了直接评估，并报告了在低并发场景下的负面加速结果，这阐明了该信号更适合用于内存而非延迟。KV缓存压缩方法（如KIVI（Liu等，2024b））减少了联合工作集的另一侧，并与WiSP组合使用而非相互竞争。最后，WiSP在MoE大语言模型（Shazeer等，2017；Fedus等，2022；Jiang等，2024；DeepSeek-AI，2024；Qwen Team，2025）、MoE视觉语言模型（Wu等，2024；Kimi Team，2025；Li等，2024）、混合SSM-MoE（Lieber等，2024；Anthony等，2024）以及用于图像（Peebles & Xie，2023；Fei等，2024）和语言（Nie等，2025；Zhu等，2025）的MoE扩散模型上进行了评估；我们仅将这些作为测试对象，未对其做任何修改。

B 动态双重调整控制律

回顾 $a = LM_e$。在已排空的屏障处，守恒预算被直接测量，$B = \kappa b + Ca$，目标大小 C，$\tilde{\kappa}$ 为

$$\tilde{\kappa} = \max\big(\kappa_{\min}, \lceil (1+h)\, p \rceil\big), \tag{6}$$

$$\tilde{C} = \mathrm{clip}\big(\lfloor (B - \tilde{\kappa}\, b)/a \rfloor,\, C_{\min},\, C_{\max}\big), \tag{7}$$

$$\tilde{\kappa} \leftarrow \lfloor (B - \tilde{C}\, a)/b \rfloor, \tag{8}$$

其中 p 是突发请求的峰值 KV 占用（实际使用的块数），h 是 KV 余量。公式 (6) 将准入下限 (4) 从一次性的启动约束提升为持续强制执行的不变量；(7) 是 KV 阶跃曲线的等边际"剩余分配给专家"规则；重计算 (8) 回收上限截断留下的任何余量，因此该操作永远不会过度提交（$\tilde{C}\, a + \tilde{\kappa}\, b \le B$）。

专家暂存区的调整会增大或缩小每层 GPU 暂存区 S 至 C 个槽位，将当前驻留的专家重新分页从 W^{cpu}，并在缩小时丢弃受害者，同时 π、π^{-1} 和 LRU 计时器也随之调整。KV 池调整会将工作进程的每层 KV 张量重新分配到 $\tilde{\kappa}$ 个块，并原地重建调度器的共享块池，保留缓存的空块标识，仅在池完全排空时执行。为将瞬态占用保持在 max(old, new) $\le B$ 而非二者之和，控制器始终先释放缩小的一侧，再增大另一侧（在专家增大时：先缩小 KV，再增大暂存区；在缩小时：相反）。每一步操作都被包裹起来，使得失败的调整会被跳过——引擎保持在先前的可用状态——而容量死区 Δ 会抑制小于 Δ 个专家的移动，从而使分割在逐轮噪声下保持稳定。

C 实现细节

WiSP 通过标准的 vllm.general 插件入口点进行注册，因此只需一次 pip install 即可；该插件会在每个 vLLM 进程中、在任何模型类被导入之前触发，并对 fused-MoE 模块上的三个方法进行补丁——权重创建、加载后处理和前向传播。权重创建在固定主机内存上分配专家主副本（这正是避免加载时 GPU 峰值的原因）；加载后处理构建每层的 C 槽暂存区以及映射 π、π^{-1}；前向传播运行算法 1，并以 π 作为专家映射来派发未经修改的内核。FP8 块量化路径会为每个专家分页四个张量（权重及其块缩放），并强制 Triton 后端绕过 DeepGEMM 的加载后处理，因为后者假定完全驻留。动态控制器的两个原语是调整层上限（扩大或缩小专家暂存区，从主机主副本重新分页）以及一次 KV 池重新分配，该分配会就地重建调度器的块池（附录 B）。

D 服务设计空间

本附录扩展了 WiSP 在现有系统（如 2.3 节所总结）中的定位。表 5 针对对我们设定至关重要的维度（路由感知、字节一致、交付及目标体制），将 WiSP 与先前的自回归 MoE-LLM 服务系统进行了对比；表 6 则列出了非自回归 MoE 模态的原生服务栈以及 WiSP 如何与每种模态对接。

E 共激活预测器

第4节使用一个轻量级路由预测器，来测试专家预测能否在低并发解码中隐藏缺页。

Table 5. AR MoE-LLM serving systems. WiSP is the low-concurrency, drop-in, reuse-caching point in the design space.

System	expert mechanism	routing-aware	byte-identical	delivery	regime
vLLM `--cpu-offload-gb` (Kwon et al., 2023)	static, layer-grained	×	✓	built-in	any (wasteful)
ktransformers (KVCache.AI, 2024)	CPU-side compute (AMX)	×	✓	engine	Intel CPU+GPU
llama.cpp (Gerganov et al., 2023)	GGUF offload	×	× (lossy)	engine	edge
MoE-Lightning (Cao et al., 2025)	pipelined paged weights	✓	✓	engine	high-batch
SiDA (Du et al., 2024)	data-aware hash offload	✓	× (approx)	engine	high-batch
FluxMoE (Liu et al., 2026) (concurrent)	stream-and-evict + planner	✓	✓	vLLM integ.	high-batch
WiSP (ours)	**cache-and-reuse via `expert_map`**	✓	✓	plug-in (entry point)	**low-concurrency**

Table 6. Serving stacks for non-AR MoE modalities and how WiSP attaches.

Modality	Representative	Native serving stack	Native expert paging	WiSP integration
MoE VLM	Kimi-VL, DeepSeek-VL2	vLLM (multimodal)	none	same engine plug-in
Hybrid SSM–MoE	Jamba	vLLM (`JambaForCausalLM`)	none; offload unsupported	same plug-in
MoE image diffusion	DiT-MoE	diffusers / repo (PyTorch)	none	∼150-line wrapper
MoE diffusion LLM	LLaDA-MoE	HF Transformers / dInfer	none	same wrapper

decode. The predictor is deliberately simple and online: it needs no gradients, calibration set, or retraining, and it uses only routing decisions that the serving engine already observes.

Construction. For each source layer ℓ and later target layer $\ell' > \ell$, the predictor keeps a co-activation table

$$A_{\ell,\ell'}(e, e'),$$

which counts how often source expert e at layer ℓ and target expert e' at layer ℓ' fire in the same forward pass. We normalize by the source marginal count to obtain an empirical conditional probability,

$$P_{\ell,\ell'}(e' \mid e) = \frac{A_{\ell,\ell'}(e, e')}{\sum_j A_{\ell,\ell'}(e, j)}.$$

At inference time, after the router has selected experts in earlier layers, the predictor scores a candidate expert e' for target layer ℓ' by summing the conditional probabilities from the experts already observed:

$$s_{\ell'}(e') = \sum_{\ell < \ell'} \gamma^{\ell' - \ell} \sum_{e \in R_\ell} P_{\ell,\ell'}(e' \mid e),$$

where $\gamma \in (0, 1]$ is a distance-decay factor. The highest-scoring experts are used as the predicted target-layer experts. The table can start empty and be learned online, or it can be initialized from a population trace or a per-user trace.

Why this is a fair prefetch test. The predictor is not intended as a new model architecture or a trained drafter. It is a low-overhead test of whether the routing signal already present during serving is useful for prefetching. It is strong enough to make the prefetch hypothesis plausible: on per-user traces it reaches nontrivial next-expert precision, around 0.5 in our Qwen3 experiments. Thus the negative result in Section 4 is not because routing is completely unpredictable. The failure mode is systems-level: even useful predictions compete with demand paging for the same PCIe bandwidth, and at tight caps the extra speculative transfers can evict experts that are still needed.

Using the same signal for memory sizing. Although the predictor does not improve single-stream decode speed, it remains useful as a memory signal. A routing trace reveals the support of the expert working set a workload repeatedly revisits. That support can be used to choose an expert cap without sweeping VRAM budgets: a global cap that is safe for all users may waste memory on most of them, while a workload-specific cap can keep the reused experts resident and return the remaining bytes to the KV cache. This is the role of routing information in WiSP: not to hide transfer latency, but to estimate the resident expert set that the allocator should budget for.

F Extended Evaluation

This appendix gives the full versions of the working-set and cross-architecture results that Section 5.4 summarizes.

F.1 The Working-Set Law, Made Measurable

Figure 4 characterizes the structure that governs where prediction *could* pay off, on a model where the working set is unusually clean to read off. A diffusion forward routes hundreds of patch tokens through the experts, so its per-forward working set is essentially the entire expert set, and the fault counts trace Denning's law exactly: below the working set the cache thrashes, and at the working set faults collapse and reuse takes over. Resident memory scales linearly with the cap, four times smaller at the tightest setting, and the output is bit-exact throughout. This is the cleanest illustration we have of *why* a single, fully-activated reference stream lets paging bound memory but not latency—and, read the other way, of why diffusion's abundant per-step compute is a regime where a prefetcher would at least have computa-

表5. AR M oE-LLM服务系统。WiSP是设计空间中低并发、即插即用、复用缓存的方案 .

系统	专家机制	路由感知	字节级完全相同	交付	机制
vLLM --cpu-offload-gb（Kwon 等，2023）	静态、层粒度	×	✓	内置	任意（浪费的）
ktransformers（KVCache.AI，2024）	CPU侧计算（AMX）	×	✓	引擎	Intel CPU+GPU
llama.cpp（Gerganov 等人，2023）	GGUF 卸载	×	×（有损）	引擎	edge
MoE-Lightning（Cao 等，2025）	流水线分页权重	✓	✓	发动机	高批次
SiDA（Du 等，2024）	数据感知哈希卸载	✓	×（近似）	引擎	高批次
FluxMoE（Liu 等，2026）（并发）	流式与逐出 + planner	✓	✓	vLLM 集成	高批处理
WiSP（本文方法）	通过专家映射进行缓存与重用	✓	✓	插件（入口点）	低并发

表 6. 非自回归 MoE 模态的服务栈以及 WiSP 如何接入。

模态	代表性	原生服务栈	原生专家分页	WiSP 集成
MoE VLM	Kimi-VL、DeepSeek-VL2	vLLM（多模态）	none	同引擎插件
混合SSM–MoE	Jamba	vLLM (JambaForCausalLM)	无；不支持卸载	同一插件
MoE图像扩散	DiT-MoE	diffusers / 仓库（PyTorch）	none	∼150-行包装器
MoE 扩散 LLM	LLaDA-MoE	HF Transformers / dInfer	none	相同的包装器

该预测器被刻意设计得简单且在线：它不需要梯度、校准集或重新训练，并且仅使用服务引擎已经观测到的路由决策。

构建。对于每个源层 ℓ 和后续目标层 $\ell' > \ell$，预测器维护一个共激活表，

$$A_{\ell,\ell'}(e, e'),$$

该表记录源专家 e 在层 ℓ 和目标专家 e' 在层 ℓ' 在同一次前向传播中共同被激活的频率。我们通过源边际计数进行归一化，以获得经验条件概率，

$$P_{\ell,\ell'}(e' \mid e) = \frac{A_{\ell,\ell'}(e, e')}{\sum_j A_{\ell,\ell'}(e, j)}.$$

在推理时，当路由器已在较早层选择了专家后，预测器通过将已观察到的专家的条件概率求和，为目标层 ℓ' 的候选专家 e'打分：

$$s_{\ell'}(e') = \sum_{\ell < \ell'} \gamma^{\ell' - \ell} \sum_{e \in R_\ell} P_{\ell,\ell'}(e' \mid e),$$

其中 $\gamma \in (0, 1]$ 是距离衰减因子。得分最高的专家被用作预测的目标层专家。该表可以从空开始在线学习，也可以从群体轨迹或用户个人轨迹初始化。

为什么这是一个公平的预取测试。该预测器并非旨在作为一种新的模型架构或经过训练的草拟器。它是对服务期间已经存在的路由信号是否对预取有用的低开销测试。它足够强大，足以使预取假设显得合理：在每用户轨迹上，它达到了非平凡的下一个专家精度，在我们的Qwen3实验中约为0.5。因此，第4节中的负面结果并不是因为路由完全不可预测。失败模式是系统级的：即使有用的预测也会与按需分页竞争相同的PCIe带宽，而在严格的容量上限下，额外的推测性传输可能会驱逐仍然需要的专家。

将相同信号用于内存规模估计。尽管预测器不能提升单流解码速度，但它作为内存信号仍然有用。路由轨迹揭示了工作负载反复访问的专家工作集的支持集。该支持集可用于在不扫描VRAM预算的情况下选择专家上限：对所有用户安全的全局上限可能会在大多数用户上浪费内存，而针对特定工作负载的上限可以将被复用的专家保持驻留，并将剩余字节归还给KV缓存。这就是路由信息在WiSP中的作用：不是隐藏传输延迟，而是估计分配器应为之预算的驻留专家集。

F 扩展评估

本附录给出了第5.4节所总结的工作集与跨架构结果的完整版本。

F.1 可度量的工作集定律

图4刻画了决定预测能否带来收益的结构，所用模型的工作集异常清晰易读。扩散前向将数百个补丁令牌路由经过专家，因此其每次前向的工作集本质上就是整个专家集，而故障计数完全遵循Denning定律：低于工作集时缓存发生抖动，达到工作集时故障骤降，复用开始主导。常驻内存随上限线性扩展，在最紧设置下缩小为四分之一，且输出全程逐位精确。这是我们拥有的最清晰的例证，说明为何单一、完全激活的参考流能使分页限制内存而非延迟——反过来读，也说明为何扩散每步充裕的计算量是一种预取器至少还有计算可隐藏的机制，

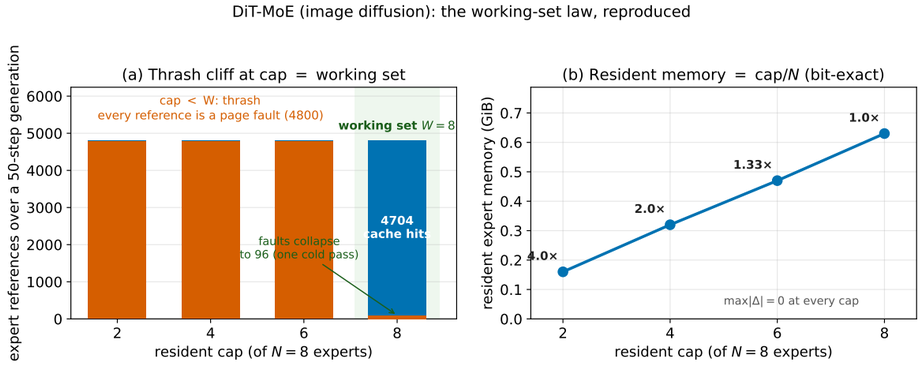


Figure 4. Working-set theory reproduced on an image-diffusion MoE (DiT-MoE-S, standalone pager). (a) Because a diffusion forward activates essentially the whole expert set over its patch tokens, a cap below the working set thrashes—every reference faults (4800 faults, 0 hits, i.e. 8 experts × 12 MoE blocks × 50 denoising steps)—and at cap = working set the faults collapse to a single cold pass (96) with 4704 hits. (b) Resident expert memory scales as cap/N, up to $4\times$ smaller, and the generation is bit-exact at every cap.

tion to hide behind, unlike single-stream LLM decode. On a fine-grained, heavily load-balanced LLM under diverse prompts (Qwen3-30B-A3B), by contrast, routing is too uniform for right-sizing to exploit and the throughput-versus-cap curve is essentially topic-independent—a conservative lower bound we report for honesty.

F.2 A Different Structure: Temporal Locality in a Diffusion LLM

DiT-MoE shows one kind of working-set structure—spatial, within a single forward, with no reuse below the working set. A diffusion *language* model exhibits the complementary kind. LLaDA-MoE generates by iteratively denoising the *same* sequence over many steps, so we ask not how a single forward routes but how routing evolves *across* denoising steps. Figure 5 makes the contrast with DiT precise. The per-step working set is large—about 96% of the 64 experts fire in every step—so a naive single-step pager thrashes just as it does on DiT, and bounding memory is again the only thing a small cap buys *within* a step. But across steps the picture changes: adjacent denoising steps agree on 44% of their per-position routing decisions, nearly twice the 23% of a shuffled-order baseline, and the agreement decays monotonically with step distance—the signature of genuine temporal locality rather than uniform reuse. DiT's structure is spatial and confined to one forward, so a prefetcher has nothing to anticipate across forwards; LLaDA's structure is temporal, so the experts a step will need are largely the ones the previous step just used, and a warm or predictive cache that survives across steps has real structure to exploit. Together these bracket the two regimes the framing predicts: a fully-activated stream with no cross-step reuse (DiT), where paging is a pure memory lever, and a fully-activated stream *with* cross-step reuse (LLaDA), where the routing signal has structure a cache or prefetcher could exploit—the diffusion counterpart of the multi-turn agent reuse we leave to future work.

G Limitations

WiSP currently runs in eager mode because the plug-in does not yet support CUDA graphs; both sides of our comparisons use eager mode for fairness. The per-layer paging decision also sits on the Python hot path. This overhead matters less in the sub-fitting regime where paging avoids large PCIe transfers, but it becomes visible as the expert cap grows and there is less to page. Moving this decision off the Python path is the most direct systems optimization. We also do not benchmark ktransformers on its Intel AMX fast path, so our comparison to that system is a regime disclosure rather than a measured loss.

Our headline iso-VRAM results for Qwen3 and Kimi-VL are measured on a physically constrained 24 GiB RTX 3090 (PCIe 4.0). This removes the emulation issue, but it also means the card cannot reach the full-footprint budget where static residency becomes faster; we characterize that crossover only on a larger GPU. The MiniMax-M2 result is likewise scoped as a capability check, not an iso-throughput win.

The MV-WSA results are also scoped. The startup-configurator result in Table 3 is a real `vllm serve` run, but it covers one KV-pressured setting. The live dual-resize controller in Table 4 is a proof of concept on real kernels: it uses the closed-form step-curve rule rather than the full

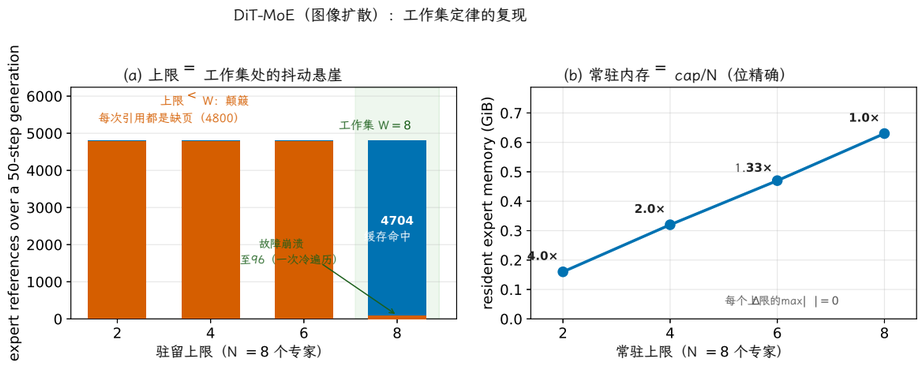


图4. 在图像扩散MoE（DiT-MoE-S，独立分页器）上复现的工作集理论。(a) 由于扩散前向过程在其patch token上基本激活了整个专家集，低于工作集的容量上限会导致颠簸——每次引用都缺页（4800次缺页，0次命中，即8个专家 × 12 MoE块 × 50 去噪步）——而在容量上限达到工作集时，缺页骤降至单次冷遍历（96次），命中4704次。(b) 常驻专家内存按cap/N缩放，最多缩小 $4\times$，且在每个容量上限下生成均为位精确。

不同于单流LLM解码。相比之下，在多样提示下运行的一个细粒度、负载高度均衡的LLM（Qwen3-30B-A3B）上，路由过于均匀，无法通过合理配置加以利用，吞吐量与上限的曲线基本上与主题无关——这是我们为诚实起见报告的保守下界。

分页纯粹是一种内存调节手段；以及完全激活且具有跨步骤复用的流（LLaDA），其中路由信号具有缓存或预取器可以利用的结构——这是我们将留待未来工作研究的多轮智能体复用在扩散模型中的对应情形。

F.2 一种不同的结构：扩散*LLM*中的时间局部性

DiT-MoE展示了一种工作集结构——在单次前向传播内、空间上的工作集结构，且工作集以下没有复用。扩散语言模型则展现出互补的一类。LLaDA-MoE通过在多个步骤中对同一序列进行迭代去噪来生成，因此我们不是问单次前向传播如何路由，而是问路由如何在去噪步骤之间演变。图5清晰地展示了与DiT的对比。每步的工作集很大——64个专家中约有96%在每一步都会激活——因此，朴素的单步分页器会像在DiT上一样频繁颠簸，而在单步内，设置小的上限所能带来的依然只是内存限制。但在步骤之间，情况发生了变化：相邻去噪步骤在其逐位置路由决策上有44%一致，几乎是打乱顺序基线23%的两倍，并且这种一致性随着步骤距离单调衰减——这是真正时间局部性而非均匀复用的标志。DiT的结构是空间性的，且局限于一次前向传播，因此预取器没有什么可跨前向传播去预判的；LLaDA的结构是时间性的，因此某一步将需要的专家在很大程度上就是上一步刚刚使用过的那些，而一个能跨步骤存活的温缓存或预测缓存就有真实的结构可供利用。两者共同界定了该框架所预测的两种情形：完全激活且没有跨步骤复用的流（DiT），其中

G 局限性

WiSP 当前在 *eager* 模式下运行，因为该插件尚不支持 CUDA graphs；为公平起见，我们比较的双方均使用 *eager* 模式。逐层的分页决策同样位于 Python 热路径上。在 sub-fitting 机制下，分页避免了大量 PCIe 传输，这一开销影响较小，但随着专家上限增长、可分页的内容减少，该开销会变得明显。将这一决策移出 Python 路径是最直接的系统优化。我们也没有在 Intel AMX 快速路径上对 ktransformers 进行基准测试，因此与该系统的比较是一种机制披露，而非实测的性能损失。

我们针对 Qwen3 和 Kimi-VL 的主要 iso-VRAM 结果是在物理受限的 24GiB RTX 3090（PCIe 4.0）上测得的。这消除了仿真问题，但也意味着该显卡无法达到静态驻留变得更快所需的完整占用预算；我们仅在更大的 GPU 上刻画了这一交叉点。MiniMax-M2 的结果同样被限定为能力验证，而非 iso-throughput 上的胜出。

MV-WSA 的结果也是有范围限定的。表 3 中的启动配置器结果是一次真实的 vllm serve 运行，但它只涵盖了一个受 KV 压力的设置。表 4 中的实时双调整控制器是在真实内核上的概念验证：它使用的是闭式阶梯曲线规则，而非完整的

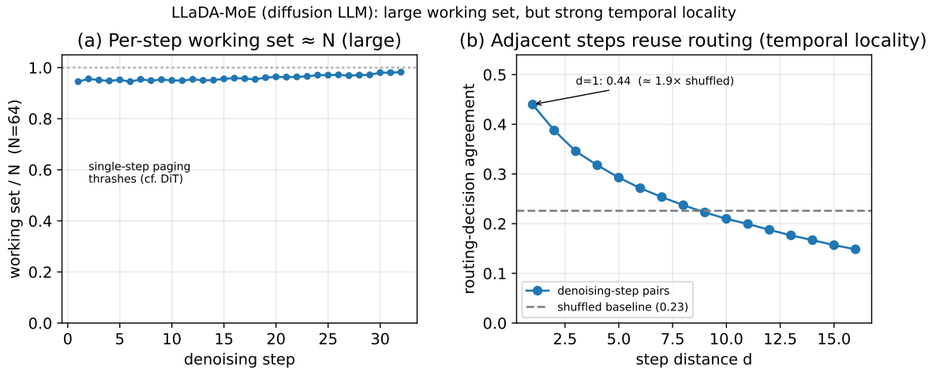


Figure 5. LLaDA-MoE has a large but temporally local working set. (a) Each denoising step activates ≈96% of all 64 experts, so—exactly as in DiT—paging a single step thrashes unless the cap is near N. (b) Yet adjacent denoising steps reuse their routing decisions: the per-position top-k agreement is 0.44 at distance one (≈1.9× a shuffled-step baseline of 0.23) and decays smoothly with step distance. The structure is temporal, not spatial.

equimarginal estimator, runs inside the in-process v1 engine rather than the multi-process `vllm serve` loop, and is evaluated on one model family and two workloads. Its byte-identity under resizing is checked by a dedicated resize-cycle test rather than a continuous regression suite. Broader workload coverage, serve-loop integration, self-calibrated cost constants, and porting the full estimator onto the live resize path are the next engineering steps.

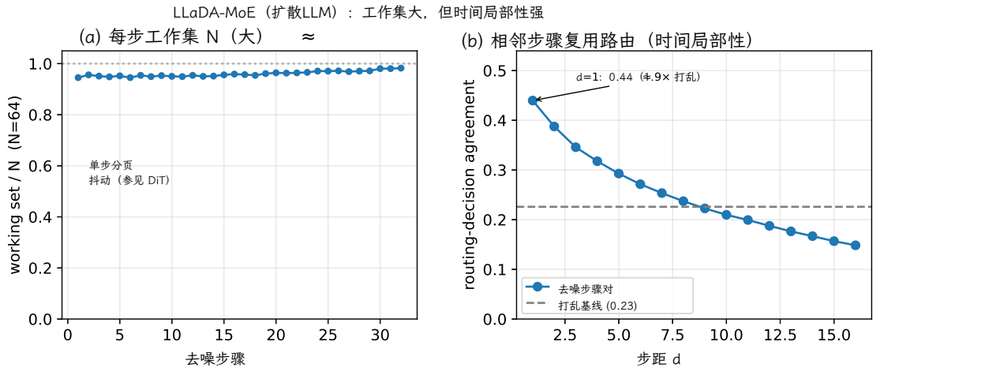


图 5. LLaDA-MoE 具有较大但在时间上局部的工作集。(a) 每个去噪步骤激活全部 64 个专家中的 ≈96%，因此——与 DiT 中完全相同——除非上限接近 N，否则单步分页会产生抖动。(b) 然而相邻去噪步骤会复用其路由决策：距离为一时每位置 top-k 一致率为 0.44（≈1.9× a 打乱步骤基线为 0.23），并随步骤距离平滑衰减。该结构是时间性的，而非空间性的。

等边际估计器，运行在进程内 v1 引擎中，而不是多进程 vllm serve 循环中，并且仅在一个模型系列和两个工作负载上进行了评估。其在调整大小下的字节一致性由专门的调整大小循环测试检查，而非持续回归套件。更广泛的工作负载覆盖、serve 循环集成、自校准成本常数，以及将完整估计器移植到实时调整路径上，是接下来的工程步骤。